

Naija Prime School

Sprint 6 — Fees, Invoices, Receipts & Bursar Workflows

Fee schedules · Invoice issuance · Payment recording · Receipts · Bursar dashboard

Long-form implementation walk-through

Author: Benjamin Fadina

Branch: sprint/6-fees-bursar

Built on: Sprints 1–5b (identity, academic domain, students & parents, attendance, results & report cards, pupil photos)

Stack: .NET 10, Blazor Web App (Auto), EF Core 10, SQL Server, Radzen Blazor

Editor: Visual Studio Code with the C# Dev Kit

Repository: <https://github.com/benjaminsqlserver/NaijaPrimeSchool>

Licence: MIT — see LICENSE at the repo root

Contents

Contents.....	2
1. Sprint 6 in context.....	6
1.1 Where this sits relative to sprint 5b.....	6
1.2 Functional scope delivered.....	7
1.3 Non-goals deliberately deferred.....	7
1.4 Scale of the sprint.....	7
2. Design decisions and trade-offs.....	9
2.1 Schedule → Invoice → Payment, three tables.....	9
2.2 No enums — four lookup tables.....	9
2.3 Decimal precision, not floats.....	9
2.4 Receipt and invoice numbering.....	9
2.5 Allocations as their own table.....	9
2.6 Soft delete, again, with guards.....	10
2.7 Unique indexes.....	10
2.8 Status recomputation on every money change.....	10
2.9 PaymentService depends on the concrete InvoiceService.....	10
2.10 Inline forms, the same UI rhythm.....	10
3. The Domain layer in full.....	11
3.1 Folder layout.....	11
3.2 The four lookup entities.....	11
3.2.1 FeeCategory.cs.....	11
3.2.2 PaymentMethod.cs.....	11
3.2.3 InvoiceStatus.cs.....	12
3.2.4 PaymentStatus.cs.....	12
3.3 The six core entities.....	13

3.3.1 FeeSchedule.cs.....	13
3.3.2 FeeScheduleItem.cs.....	13
3.3.3 Invoice.cs.....	13
3.3.4 InvoiceLine.cs.....	14
3.3.5 Payment.cs.....	15
3.3.6 PaymentAllocation.cs.....	15
3.4 Relationships at a glance.....	16
4. Application layer — DTOs and contracts.....	17
4.1 FeeScheduleDtos.cs.....	17
4.2 InvoiceDtos.cs.....	18
4.3 PaymentDtos.cs.....	20
4.4 StudentLedgerDtos.cs.....	22
4.5 Service contracts.....	23
4.6 ILookupService extension.....	25
5. Infrastructure — DbContext changes.....	26
5.1 Ten new DbSets.....	26
5.2 ConfigureFinance.....	26
6. Infrastructure — service implementations.....	31
6.1 FeeScheduleService.cs.....	31
6.2 InvoiceService.cs.....	35
6.3 PaymentService.cs.....	43
6.4 DI registration.....	49
6.5 LookupService — four new methods.....	49
7. The EF Core migration.....	51
8. Seeding the finance lookups.....	63
9. The Razor pages.....	66

9.1 FeeSchedules.razor — the schedule list.....	66
9.2 FeeScheduleDetail.razor — schedule + items.....	72
9.3 Invoices.razor — the invoice list.....	79
9.4 IssueInvoices.razor — fan-out from a schedule.....	82
9.5 InvoiceDetail.razor — the invoice view.....	85
9.6 Payments.razor — the receipts list.....	89
9.7 RecordPayment.razor — the bursar's collection workflow.....	92
9.8 PaymentDetail.razor — the receipt view.....	97
9.9 FinanceDashboard.razor — bursar at a glance.....	101
10. Navigation, imports, and authorization.....	105
11. Lifecycle of a finance flow.....	106
11.1 Publishing a fee schedule.....	106
11.2 Issuing invoices.....	106
11.3 Recording a payment.....	106
11.4 Refunding a bounced cheque.....	106
12. Smoke-test walkthrough.....	107
12.1 Build, migrate, run.....	107
12.2 Verify navigation and lookups.....	107
12.3 End-to-end happy path.....	107
12.4 Error paths.....	107
13. Troubleshooting and gotchas.....	108
13.1 'A fee schedule already exists for this (term, class level)'.....	108
13.2 'Publish the fee schedule before issuing invoices from it'.....	108
13.3 'No actively-enrolled pupils in the selected class'.....	108
13.4 Allocations exceed the payment amount.....	108
13.5 Cannot delete a payment.....	108

13.6 'Invoice statuses are not seeded' / 'Cancelled status is not seeded'	108
13.7 The IdentityRole migration warning.....	108
14. Forward-compatibility, today.....	109
15. Appendix — files added or changed in sprint 6.....	110

1. Sprint 6 in context

Sprint 6 lays the financial spine of the school. The first five sprints established who the people are, what the calendar looks like, which pupil is in which class, who showed up on which day, and how each pupil performed. Sprint 6 turns its attention to the exchange of money that pays for all of that: the fee schedule the head teacher publishes at the start of every term, the per-pupil invoice that drops into the bursar's queue when the term begins, the cash, transfer, POS, cheque, mobile-money or online payment that comes back in, and the receipts that go home in the school bag.

Functionally, the sprint introduces a three-stage pipeline. Stage one is the schedule: a SuperAdmin or HeadTeacher (or the bursar herself) creates one FeeSchedule per (Term, ClassLevel) and lists its line items, each one keyed to a FeeCategory (Tuition, Development Levy, Examination, Books, Uniforms, Transport, Meals, Boarding, PTA Levy, Other) with a published amount. Stage two is issuance: a single click on a published schedule fans out into one Invoice per actively-enrolled pupil in the chosen class, each invoice carrying a copy of the schedule's items so the bill stays stable even if the schedule is later refined. Stage three is collection: the bursar records a Payment against the pupil, allocates it across one or more outstanding invoices, and the system stamps the receipt number, updates each invoice's status, and produces a printable receipt view.

Once this sprint ships, the school has a complete-enough money loop to operate without spreadsheets. The bursar dashboard summarises total invoiced, total collected, total outstanding, and a per-method and per-category breakdown for the active term. The Family → Students profile and the Report Card detail page will, in a later sprint, surface the same data on the pupil side; the model is already in place.

This document is a long-form implementation guide in the tone of the sprint 5 guide. An engineer who has read sprints 1–5b and has the codebase checked out can recreate every change here without referring to the diff. The structure mirrors the build order: design decisions first, Domain entities next, Application contracts after that, Infrastructure (DbContext, services, seeder, migration) in the middle, then the Razor UI and navigation. Smoke-test, troubleshooting, and forward-compatibility chapters round it off.

1.1 Where this sits relative to sprint 5b

Every load-bearing piece of the earlier sprints is reused:

- BaseEntity — every new entity in this sprint inherits Guid Id, IAuditable, and ISoftDelete from it.
- ApplicationDbContext.SaveChanges — the override stamps audit columns and rewrites Delete to IsDeleted = true. The finance rows therefore inherit the same audit and soft-delete machinery as everything else.
- Global query filters — every new entity declares HasQueryFilter(x => !x.IsDeleted), so deleted rows vanish from ordinary queries automatically.
- OperationResult / OperationResult<T> — every new service uses this for predictable success/failure responses.
- ILookupService — already had twenty methods. Sprint 6 adds four more (fee categories, payment methods, invoice statuses, payment statuses) without rewriting the existing ones.
- Student, Term, SchoolClass, ClassLevel — pick up collection navigations only. No scalar columns change on any of those tables, so existing pages still render unchanged.

- StudentAvatar (sprint 5b) — already lives in Components.Shared and is reused on every invoice and payment row so a pupil's face follows them through the finance views.

1.2 Functional scope delivered

1. Create a FeeSchedule for any (Term, ClassLevel), give it a title and optional notes, then add line items keyed to a FeeCategory.
2. Publish a schedule. Once published, the schedule and its items are locked from edits; unpublishing is allowed only while no invoices have been issued from it.
3. From the Issue Invoices page, pick a published schedule and a matching class. The system creates one Invoice per actively-enrolled pupil with a copy of every schedule line item, assigns a sequential InvoiceNumber (NPS/INV/<year>/<seq>), and stamps the issued-on date.
4. Apply a per-line discount on an invoice. Subtotal, AmountDue and the invoice status are recomputed automatically.
5. Record a Payment for a pupil, pick a method (Cash, Bank Transfer, POS, Cheque, Mobile Money, Online Payment), enter the amount, optionally an auto-allocate across outstanding invoices oldest-first, or hand-allocate row-by-row.
6. Open a payment to see the receipt: receipt number, method, reference, allocations, and the running balance each invoice moved to.
7. Refund a payment. Allocations are released, the receipt stays on file with a 'Refunded' badge, and the touched invoices flip back to their pre-payment state.
8. Open the Bursar dashboard for a term to see total invoiced, total collected, total outstanding, and the breakdown of collections by payment method and invoiced amounts by fee category.
9. Soft-delete any of the above with guards: cannot delete a schedule that has issued invoices, cannot delete an invoice with payments applied, cannot delete a payment that still has allocations.

1.3 Non-goals deliberately deferred

- Printable PDF invoices and receipts. The DTOs carry every field a layout would need; the Razor pages already render a human-readable receipt view. A QuestPDF-driven endpoint is a follow-on lift.
- Payment gateway integration. Cards, USSD, and online checkout would each plug in as a new PaymentMethod row plus a webhook endpoint. The internal model — Payment + PaymentAllocation — is ready to receive callbacks unchanged.
- Multi-currency. Amounts are stored as decimal(12,2). The school is single-currency (NGN) today; a Currency lookup would be added by future maintainers without schema gymnastics.
- Scholarships and waivers as a first-class concept. Today the discount column on InvoiceLine handles per-pupil reductions; a dedicated Scholarship table could later be introduced.
- Parent / student visibility. Parents and pupils still see placeholder navigation. The portal sprint will expose existing invoices and the unallocated payments using the Invoice.Notes + Payment.ReceiptNumber the bursar already produces.
- Aged-debt reports and dunning notices. The data the report needs (DueDate + Balance per pupil) is in place; the report itself is a feature-add.

1.4 Scale of the sprint

By the numbers:

- 10 new domain entities under `src/NaijaPrimeSchool.Domain/Finance/`.
- 4 collection navigations on existing entities (Student, Term, SchoolClass, ClassLevel).
- 4 DTO files under `src/NaijaPrimeSchool.Application/Finance/Dtos/`.
- 3 new service contracts under `src/NaijaPrimeSchool.Application/Finance/`.
- 3 service implementations under `src/NaijaPrimeSchool.Infrastructure/Services/`.
- 4 new methods on `ILookupService` (and the matching `LookupService`).
- 1 EF Core migration introducing 10 new tables and the indexes that go with them.
- 1 `DatabaseInitializer` extension seeding `FeeCategories`, `PaymentMethods`, `InvoiceStatuses` and `PaymentStatuses`.
- 8 Razor pages under `src/NaijaPrimeSchool.Web/Components/Pages/Finance/`.
- 1 navigation menu rewrite: the previously-disabled 'Finance' placeholder is replaced with a six-item panel gated to SuperAdmin + HeadTeacher + SchoolBursar.

Everything compiles with zero warnings on .NET 10. The code follows the patterns already accepted in sprints 1–5b.

2. Design decisions and trade-offs

2.1 Schedule → Invoice → Payment, three tables

The pipeline could in theory be collapsed: 'the bursar enters a payment, the system computes the bill from a schedule, sends the receipt.' Real schools do not work that way. The bill goes home in week one of term so the family can plan; the payment trickles in over weeks; receipts go back as evidence. Persisting all three stages — schedule, invoice, payment — costs four extra tables (schedule, item, line, allocation) and earns back, in exchange:

- **Stability.** A schedule edited after invoices have been issued does not retroactively change the invoiced amounts.
- **Auditability.** Each invoice carries a copy of the schedule item it came from, plus a `FeeScheduleItemId` pointing back at the source — so a future audit can answer 'where did this line come from?'
- **Bookkeeping.** A payment can sit unallocated as credit on the pupil's ledger, or fan across several invoices via `PaymentAllocation` rows; the schema does not force the bursar into a one-to-one shape that real life does not respect.

2.2 No enums — four lookup tables

The rule from earlier sprints holds. `FeeCategory`, `PaymentMethod`, `InvoiceStatus`, and `PaymentStatus` are all proper entities derived from `BaseEntity`, seeded on first run, and editable from the database without a redeploy. Each one carries a `Code` column (TUI, CASH, ISSUED, CONFIRMED, ...) that the service layer keys off when it transitions a row between states — so renaming 'Issued' to 'Active' in the row would not break logic.

2.3 Decimal precision, not floats

Every money column is `decimal(12,2)` — twelve significant digits before the decimal point is plenty for a single school's invoices (₦9,999,999,999.99) and two after preserves kobo resolution. EF Core's `HasPrecision` attribute is used on every `Amount`, `Discount`, `AmountDue`, `AmountPaid`, and `AmountApplied` column. Float would have introduced rounding errors that compound through allocations.

2.4 Receipt and invoice numbering

Both `InvoiceNumber` and `ReceiptNumber` follow a year-prefixed pattern: `NPS/INV/<year>/<4-digit-sequence>` and `NPS/RCP/<year>/<4-digit-sequence>`. The services compute the next sequence by reading the largest existing number for the year and adding one. The cost is one extra query per issuance run; the benefit is that auditors can read a number and know, without lookups, which year it belongs to. The format is constants in the service classes and easy to change.

2.5 Allocations as their own table

`PaymentAllocation` is a separate entity with a unique (`PaymentId`, `InvoiceId`) index. A payment can apply to many invoices; an invoice can be paid down by many payments. The alternative — putting an `InvoiceId` column on `Payment` — does not allow either, and falls apart the first time a parent pays ₦100,000 toward two children's fees in one transfer.

2.6 Soft delete, again, with guards

Every new entity implements `ISoftDelete` via `BaseEntity`. What is new is the operation guards:

- `FeeSchedule` cannot be edited or items added/removed while it is published. Unpublish first.
- `FeeSchedule` cannot be unpublished if any invoice has ever been issued from it — the auditable link from invoice line back to the schedule item would otherwise become misleading.
- `FeeSchedule` cannot be deleted if it has been used to issue invoices.
- Invoice cannot be edited (lines, discounts) once cancelled.
- Invoice cannot be cancelled while payments are applied; refund those first.
- Invoice cannot be deleted while payments are applied.
- Payment cannot be deleted while it has allocations; refund first so the allocations are released, then delete.

2.7 Unique indexes

- Unique (`TermId`, `ClassLevelId`) on `FeeSchedule` — at most one schedule per (term, level). A school cannot accidentally publish two competing schedules.
- Unique `InvoiceNumber`, unique `ReceiptNumber` — auditable.
- Unique (`PaymentId`, `InvoiceId`) on `PaymentAllocation` — a payment cannot 'double-pay' the same invoice in a single transaction.
- Plus unique `Name/Code` on every lookup table.

2.8 Status recomputation on every money change

`InvoiceService.RecomputeInvoiceTotalsAsync` is the single seam where `Subtotal`, `DiscountTotal`, `AmountDue`, `AmountPaid`, and `InvoiceStatusId` are brought into agreement with the underlying lines and allocations. It is called from `SetLineDiscountAsync` after a discount changes and from `PaymentService.RecordAsync` / `RefundAsync` after allocations are created or released. A CANCELLED invoice is never automatically demoted from cancellation — the override is explicit in the helper.

2.9 PaymentService depends on the concrete InvoiceService

`RecomputeInvoiceTotalsAsync` is an internal helper, not part of the `IInvoiceService` contract. `PaymentService` takes the concrete `InvoiceService` in its constructor so it can call the helper directly. DI registers `InvoiceService` twice — once as the concrete class for `PaymentService`, once behind `IInvoiceService` for the UI. The pattern keeps the public service surface small without forcing a public 'Recompute' method that any caller could misuse.

2.10 Inline forms, the same UI rhythm

Every Razor page in this sprint follows the same shape as the earlier sprints: a filter card on top, a paged grid in the middle, an inline `RadzenCard` form below the grid for new-or-edit, and a confirm-then-act dialog for destructive operations. Two pages — `IssueInvoices` and `RecordPayment` — are dedicated full-page forms because their workflows have enough fields and side-effects that an inline form would have been cramped.

3. The Domain layer in full

Every sprint-6 entity lives in a single new folder, `src/NaijaPrimeSchool.Domain/Finance/`. Entities are anaemic — no domain methods, no validation logic. Validation lives in DTOs (DataAnnotations) and services.

3.1 Folder layout

```
src/NaijaPrimeSchool.Domain/
├── Finance/                                <- (new in sprint 6)
│   ├── FeeCategory.cs                    <- lookup
│   ├── PaymentMethod.cs                 <- lookup
│   ├── InvoiceStatus.cs                  <- lookup
│   ├── PaymentStatus.cs                 <- lookup
│   ├── FeeSchedule.cs                   <- per (term, class level)
│   ├── FeeScheduleItem.cs               <- line item on a schedule
│   ├── Invoice.cs                        <- bill issued to a pupil
│   ├── Invoiceline.cs                   <- line item on an invoice
│   ├── Payment.cs                       <- receipt header
│   └── PaymentAllocation.cs              <- payment x invoice split
├── Results/                             <- from sprint 5
├── Attendance/                           <- from sprint 4
├── Family/                               <- from sprint 3
├── Academics/                            <- from sprint 2
├── Common/                               <- from sprint 1
└── Identity/                             <- from sprint 1
```

3.2 The four lookup entities

3.2.1 FeeCategory.cs

Listing — `src/NaijaPrimeSchool.Domain/Finance/FeeCategory.cs`

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Finance;

public class FeeCategory : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public string Code { get; set; } = string.Empty;
    public int DisplayOrder { get; set; }
    public bool IsMandatoryByDefault { get; set; }

    public ICollection<FeeScheduleItem> ScheduleItems { get; set; } = [];
    public ICollection<Invoiceline> Invoicelines { get; set; } = [];
}
```

3.2.2 PaymentMethod.cs

Listing — `src/NaijaPrimeSchool.Domain/Finance/PaymentMethod.cs`

```
using NaijaPrimeSchool.Domain.Common;
```

```

namespace NaijaPrimeSchool.Domain.Finance;

public class PaymentMethod : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public string Code { get; set; } = string.Empty;
    public int DisplayOrder { get; set; }
    public bool RequiresReference { get; set; }

    public ICollection<Payment> Payments { get; set; } = [];
}

```

3.2.3 InvoiceStatus.cs

Listing — src/NaijaPrimeSchool.Domain/Finance/InvoiceStatus.cs

```

using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Finance;

public class InvoiceStatus : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public string Code { get; set; } = string.Empty;
    public int DisplayOrder { get; set; }

    // Marks the rows used during the lifecycle:
    // Draft, Issued, PartiallyPaid, Paid, Overdue, Cancelled.
    // The service layer keys off Code rather than Id when transitioning
    // an invoice between states so a row rename does not break logic.

    public ICollection<Invoice> Invoices { get; set; } = [];
}

```

3.2.4 PaymentStatus.cs

Listing — src/NaijaPrimeSchool.Domain/Finance/PaymentStatus.cs

```

using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Finance;

public class PaymentStatus : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public string Code { get; set; } = string.Empty;
    public int DisplayOrder { get; set; }

    public ICollection<Payment> Payments { get; set; } = [];
}

```

3.3 The six core entities

3.3.1 FeeSchedule.cs

Listing — src/NaijaPrimeSchool.Domain/Finance/FeeSchedule.cs

```
using NaijaPrimeSchool.Domain.Academics;
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Finance;

public class FeeSchedule : BaseEntity
{
    public Guid TermId { get; set; }
    public Term? Term { get; set; }

    public Guid ClassLevelId { get; set; }
    public ClassLevel? ClassLevel { get; set; }

    public string Title { get; set; } = string.Empty;
    public string? Notes { get; set; }

    public bool IsPublished { get; set; }
    public DateTimeOffset? PublishedOn { get; set; }

    public ICollection<FeeScheduleItem> Items { get; set; } = [];
}
```

3.3.2 FeeScheduleItem.cs

Listing — src/NaijaPrimeSchool.Domain/Finance/FeeScheduleItem.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Finance;

public class FeeScheduleItem : BaseEntity
{
    public Guid FeeScheduleId { get; set; }
    public FeeSchedule? FeeSchedule { get; set; }

    public Guid FeeCategoryId { get; set; }
    public FeeCategory? FeeCategory { get; set; }

    public string Description { get; set; } = string.Empty;
    public decimal Amount { get; set; }

    public bool IsMandatory { get; set; } = true;
    public int DisplayOrder { get; set; }
}
```

3.3.3 Invoice.cs

Listing — src/NaijaPrimeSchool.Domain/Finance/Invoice.cs

```

using NaijaPrimeSchool.Domain.Academics;
using NaijaPrimeSchool.Domain.Common;
using NaijaPrimeSchool.Domain.Family;

namespace NaijaPrimeSchool.Domain.Finance;

public class Invoice : BaseEntity
{
    public Guid StudentId { get; set; }
    public Student? Student { get; set; }

    public Guid TermId { get; set; }
    public Term? Term { get; set; }

    public Guid SchoolClassId { get; set; }
    public SchoolClass? SchoolClass { get; set; }

    public Guid InvoiceStatusId { get; set; }
    public InvoiceStatus? InvoiceStatus { get; set; }

    public string InvoiceNumber { get; set; } = string.Empty;
    public DateOnly IssuedOn { get; set; }
    public DateOnly? DueDate { get; set; }

    public decimal Subtotal { get; set; }
    public decimal DiscountTotal { get; set; }
    public decimal AmountDue { get; set; }
    public decimal AmountPaid { get; set; }

    public string? Notes { get; set; }

    public ICollection<InvoiceLine> Lines { get; set; } = [];
    public ICollection<PaymentAllocation> Allocations { get; set; } = [];

    public decimal Balance => AmountDue - AmountPaid;
}

```

Balance is a computed property — not persisted. It is just AmountDue minus AmountPaid for the convenience of callers; the persisted Subtotal/DiscountTotal/AmountDue/AmountPaid fields are kept in sync by `RecomputeInvoiceTotalsAsync`.

3.3.4 InvoiceLine.cs

Listing — src/NaijaPrimeSchool.Domain/Finance/InvoiceLine.cs

```

using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Finance;

public class InvoiceLine : BaseEntity
{
    public Guid InvoiceId { get; set; }
    public Invoice? Invoice { get; set; }
}

```

```

    public Guid FeeCategoryId { get; set; }
    public FeeCategory? FeeCategory { get; set; }

    public Guid? FeeScheduleItemId { get; set; }
    public FeeScheduleItem? FeeScheduleItem { get; set; }

    public string Description { get; set; } = string.Empty;
    public decimal Amount { get; set; }
    public decimal Discount { get; set; }

    public decimal LineTotal => Amount - Discount;
}

```

3.3.5 Payment.cs

Listing — src/NaijaPrimeSchool.Domain/Finance/Payment.cs

```

using NaijaPrimeSchool.Domain.Common;
using NaijaPrimeSchool.Domain.Family;
using NaijaPrimeSchool.Domain.Identity;

namespace NaijaPrimeSchool.Domain.Finance;

public class Payment : BaseEntity
{
    public Guid StudentId { get; set; }
    public Student? Student { get; set; }

    public Guid PaymentMethodId { get; set; }
    public PaymentMethod? PaymentMethod { get; set; }

    public Guid PaymentStatusId { get; set; }
    public PaymentStatus? PaymentStatus { get; set; }

    public string ReceiptNumber { get; set; } = string.Empty;
    public DateOnly PaidOn { get; set; }
    public decimal Amount { get; set; }

    public string? Reference { get; set; }
    public string? Notes { get; set; }

    public Guid? CollectedById { get; set; }
    public ApplicationUser? CollectedBy { get; set; }

    public ICollection<PaymentAllocation> Allocations { get; set; } = [];
}

```

3.3.6 PaymentAllocation.cs

Listing — src/NaijaPrimeSchool.Domain/Finance/PaymentAllocation.cs

```

using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Finance;

```

```

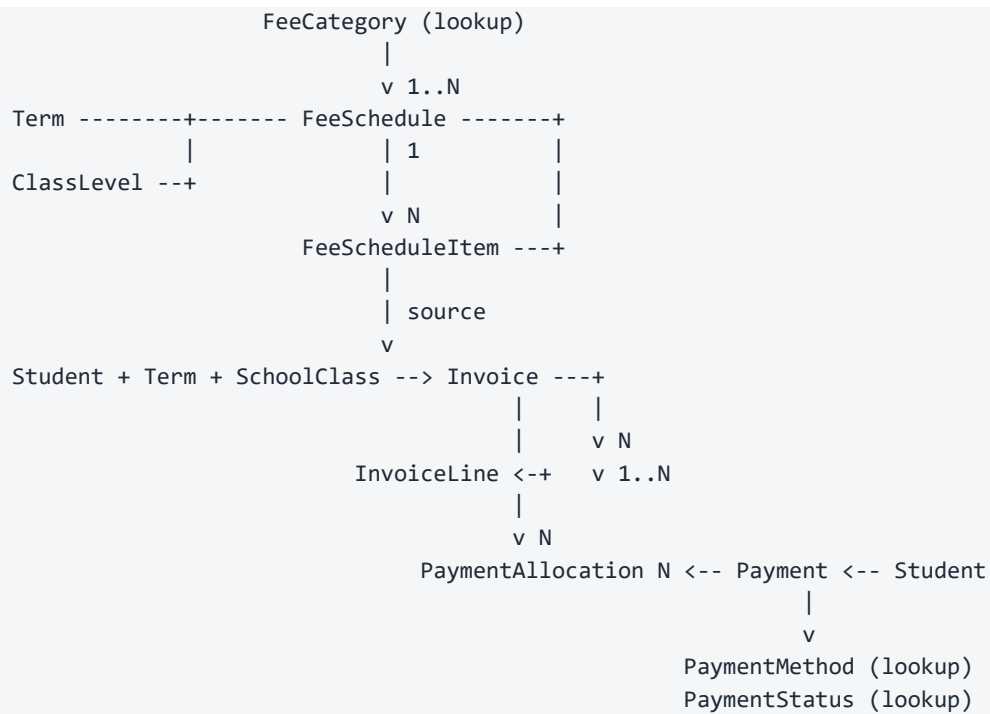
public class PaymentAllocation : BaseEntity
{
    public Guid PaymentId { get; set; }
    public Payment? Payment { get; set; }

    public Guid InvoiceId { get; set; }
    public Invoice? Invoice { get; set; }

    public decimal AmountApplied { get; set; }
}

```

3.4 Relationships at a glance



4. Application layer — DTOs and contracts

4.1 FeeScheduleDtos.cs

Listing — src/NaijaPrimeSchool.Application/Finance/Dtos/FeeScheduleDtos.cs

```
using System.ComponentModel.DataAnnotations;

namespace NaijaPrimeSchool.Application.Finance.Dtos;

public class FeeScheduleDto
{
    public Guid Id { get; set; }

    public Guid TermId { get; set; }
    public string TermName { get; set; } = string.Empty;

    public Guid SessionId { get; set; }
    public string SessionName { get; set; } = string.Empty;

    public Guid ClassLevelId { get; set; }
    public string ClassLevelName { get; set; } = string.Empty;

    public string Title { get; set; } = string.Empty;
    public string? Notes { get; set; }

    public bool IsPublished { get; set; }
    public DateTimeOffset? PublishedOn { get; set; }

    public int ItemCount { get; set; }
    public decimal TotalAmount { get; set; }
    public int InvoicesIssued { get; set; }
}

public class FeeScheduleItemDto
{
    public Guid Id { get; set; }
    public Guid FeeScheduleId { get; set; }
    public Guid FeeCategoryId { get; set; }
    public string FeeCategoryName { get; set; } = string.Empty;
    public string FeeCategoryCode { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public decimal Amount { get; set; }
    public bool IsMandatory { get; set; }
    public int DisplayOrder { get; set; }
}

public class FeeScheduleDetailDto
{
    public FeeScheduleDto Schedule { get; set; } = new();
    public List<FeeScheduleItemDto> Items { get; set; } = [];
}

public class CreateFeeScheduleRequest
```

```

{
    [Required] public Guid TermId { get; set; }
    [Required] public Guid ClassLevelId { get; set; }

    [Required, StringLength(120)]
    public string Title { get; set; } = string.Empty;

    [StringLength(500)]
    public string? Notes { get; set; }
}

public class UpdateFeeScheduleRequest
{
    public Guid Id { get; set; }

    [Required, StringLength(120)]
    public string Title { get; set; } = string.Empty;

    [StringLength(500)]
    public string? Notes { get; set; }
}

public class UpsertFeeScheduleItemRequest
{
    public Guid? Id { get; set; }
    [Required] public Guid FeeScheduleId { get; set; }
    [Required] public Guid FeeCategoryId { get; set; }

    [Required, StringLength(160)]
    public string Description { get; set; } = string.Empty;

    [Range(0.01, 100000000.0)]
    public decimal Amount { get; set; }

    public bool IsMandatory { get; set; } = true;
    public int DisplayOrder { get; set; }
}

public class FeeScheduleFilter
{
    public Guid? TermId { get; set; }
    public Guid? ClassLevelId { get; set; }
    public bool? IsPublished { get; set; }
}

```

4.2 InvoiceDtos.cs

Listing — src/NaijaPrimeSchool.Application/Finance/Dtos/InvoiceDtos.cs

```

using System.ComponentModel.DataAnnotations;

namespace NaijaPrimeSchool.Application.Finance.Dtos;

public class InvoiceDto

```

```

{
    public Guid Id { get; set; }

    public string InvoiceNumber { get; set; } = string.Empty;
    public DateOnly IssuedOn { get; set; }
    public DateOnly? DueDate { get; set; }

    public Guid StudentId { get; set; }
    public string StudentName { get; set; } = string.Empty;
    public string StudentAdmissionNumber { get; set; } = string.Empty;
    public string? StudentPhotoUrl { get; set; }
    public string StudentFirstName { get; set; } = string.Empty;
    public string StudentLastName { get; set; } = string.Empty;

    public Guid TermId { get; set; }
    public string TermName { get; set; } = string.Empty;
    public Guid SessionId { get; set; }
    public string SessionName { get; set; } = string.Empty;

    public Guid SchoolClassId { get; set; }
    public string SchoolClassName { get; set; } = string.Empty;

    public Guid InvoiceStatusId { get; set; }
    public string InvoiceStatusName { get; set; } = string.Empty;
    public string InvoiceStatusCode { get; set; } = string.Empty;

    public decimal Subtotal { get; set; }
    public decimal DiscountTotal { get; set; }
    public decimal AmountDue { get; set; }
    public decimal AmountPaid { get; set; }
    public decimal Balance => AmountDue - AmountPaid;

    public string? Notes { get; set; }
}

public class InvoicelineDto
{
    public Guid Id { get; set; }
    public Guid InvoiceId { get; set; }
    public Guid FeeCategoryId { get; set; }
    public string FeeCategoryName { get; set; } = string.Empty;
    public string Description { get; set; } = string.Empty;
    public decimal Amount { get; set; }
    public decimal Discount { get; set; }
    public decimal LineTotal => Amount - Discount;
}

public class InvoicePaymentDto
{
    public Guid PaymentId { get; set; }
    public string ReceiptNumber { get; set; } = string.Empty;
    public DateOnly PaidOn { get; set; }
    public string PaymentMethodName { get; set; } = string.Empty;
    public decimal AmountApplied { get; set; }
}

```

```

        public string? Reference { get; set; }
    }

    public class InvoiceDetailDto
    {
        public InvoiceDto Invoice { get; set; } = new();
        public List<InvoiceLineDto> Lines { get; set; } = [];
        public List<InvoicePaymentDto> Payments { get; set; } = [];
    }

    public class IssueInvoicesRequest
    {
        [Required] public Guid FeeScheduleId { get; set; }
        [Required] public Guid SchoolClassId { get; set; }
        [Required] public DateOnly IssuedOn { get; set; }
        public DateOnly? DueDate { get; set; }
        public List<Guid>? StudentIds { get; set; } // optional – null means every active
        enrolment
    }

    public class IssueInvoicesResponse
    {
        public int Issued { get; set; }
        public int Skipped { get; set; }
        public List<string> Warnings { get; set; } = [];
    }

    public class UpdateInvoiceLineDiscountRequest
    {
        public Guid InvoiceLineId { get; set; }

        [Range(0, 100000000.0)]
        public decimal Discount { get; set; }
    }

    public class InvoiceFilter
    {
        public Guid? StudentId { get; set; }
        public Guid? TermId { get; set; }
        public Guid? SessionId { get; set; }
        public Guid? SchoolClassId { get; set; }
        public Guid? InvoiceStatusId { get; set; }
        public string? Search { get; set; }
    }
}

```

InvoiceDto carries the sprint-5b photo plumbing (StudentPhotoUrl, StudentFirstName, StudentLastName) so the <StudentAvatar /> component renders pupil photographs on the invoice list and detail without a second round-trip.

4.3 PaymentDtos.cs

Listing — src/NaijaPrimeSchool.Application/Finance/Dtos/PaymentDtos.cs

```
using System.ComponentModel.DataAnnotations;
```

```

namespace NaijaPrimeSchool.Application.Finance.Dtos;

public class PaymentDto
{
    public Guid Id { get; set; }

    public string ReceiptNumber { get; set; } = string.Empty;
    public DateOnly PaidOn { get; set; }
    public decimal Amount { get; set; }

    public Guid StudentId { get; set; }
    public string StudentName { get; set; } = string.Empty;
    public string StudentAdmissionNumber { get; set; } = string.Empty;
    public string? StudentPhotoUrl { get; set; }
    public string StudentFirstName { get; set; } = string.Empty;
    public string StudentLastName { get; set; } = string.Empty;

    public Guid PaymentMethodId { get; set; }
    public string PaymentMethodName { get; set; } = string.Empty;

    public Guid PaymentStatusId { get; set; }
    public string PaymentStatusName { get; set; } = string.Empty;
    public string PaymentStatusCode { get; set; } = string.Empty;

    public string? Reference { get; set; }
    public string? Notes { get; set; }

    public Guid? CollectedById { get; set; }
    public string? CollectedByName { get; set; }

    public decimal AmountAllocated { get; set; }
    public decimal AmountUnallocated => Amount - AmountAllocated;
}

public class PaymentAllocationDto
{
    public Guid Id { get; set; }
    public Guid PaymentId { get; set; }
    public Guid InvoiceId { get; set; }
    public string InvoiceNumber { get; set; } = string.Empty;
    public decimal AmountApplied { get; set; }
    public decimal InvoiceBalanceAfter { get; set; }
}

public class PaymentDetailDto
{
    public PaymentDto Payment { get; set; } = new();
    public List<PaymentAllocationDto> Allocations { get; set; } = [];
}

public class AllocationLineRequest
{
    [Required] public Guid InvoiceId { get; set; }
}

```

```

        [Range(0.0, 100000000.0)]
        public decimal AmountApplied { get; set; }
    }

    public class RecordPaymentRequest
    {
        [Required] public Guid StudentId { get; set; }
        [Required] public Guid PaymentMethodId { get; set; }
        [Required] public DateOnly PaidOn { get; set; }

        [Range(0.01, 100000000.0)]
        public decimal Amount { get; set; }

        [StringLength(120)]
        public string? Reference { get; set; }

        [StringLength(300)]
        public string? Notes { get; set; }

        public Guid? CollectedById { get; set; }

        public List<AllocationLineRequest> Allocations { get; set; } = [];
    }

    public class UpdatePaymentRequest
    {
        public Guid Id { get; set; }

        [StringLength(120)]
        public string? Reference { get; set; }

        [StringLength(300)]
        public string? Notes { get; set; }
    }

    public class PaymentFilter
    {
        public Guid? StudentId { get; set; }
        public Guid? PaymentMethodId { get; set; }
        public Guid? PaymentStatusId { get; set; }
        public DateOnly? FromDate { get; set; }
        public DateOnly? ToDate { get; set; }
        public string? Search { get; set; }
    }
}

```

4.4 StudentLedgerDtos.cs

Listing — src/NaijaPrimeSchool.Application/Finance/Dtos/StudentLedgerDtos.cs

```

namespace NaijaPrimeSchool.Application.Finance.Dtos;

public class StudentLedgerDto
{

```

```

    public Guid StudentId { get; set; }
    public string StudentName { get; set; } = string.Empty;
    public string StudentAdmissionNumber { get; set; } = string.Empty;
    public string? StudentPhotoUrl { get; set; }
    public string StudentFirstName { get; set; } = string.Empty;
    public string StudentLastName { get; set; } = string.Empty;

    public decimal TotalInvoiced { get; set; }
    public decimal TotalPaid { get; set; }
    public decimal OutstandingBalance => TotalInvoiced - TotalPaid;

    public List<InvoiceDto> Invoices { get; set; } = [];
    public List<PaymentDto> Payments { get; set; } = [];
}

public class FinanceSummaryDto
{
    public Guid? TermId { get; set; }
    public string? TermName { get; set; }

    public int InvoiceCount { get; set; }
    public int PaidInvoiceCount { get; set; }
    public int PartiallyPaidInvoiceCount { get; set; }
    public int UnpaidInvoiceCount { get; set; }

    public decimal TotalInvoiced { get; set; }
    public decimal TotalCollected { get; set; }
    public decimal TotalOutstanding { get; set; }

    public int PaymentCount { get; set; }
    public List<MethodBreakdownDto> ByMethod { get; set; } = [];
    public List<CategoryBreakdownDto> ByCategory { get; set; } = [];
}

public class MethodBreakdownDto
{
    public Guid PaymentMethodId { get; set; }
    public string PaymentMethodName { get; set; } = string.Empty;
    public decimal TotalAmount { get; set; }
    public int PaymentCount { get; set; }
}

public class CategoryBreakdownDto
{
    public Guid FeeCategoryId { get; set; }
    public string FeeCategoryName { get; set; } = string.Empty;
    public decimal TotalInvoiced { get; set; }
}

```

4.5 Service contracts

Listing — src/NaijaPrimeSchool.Application/Finance/IFeeScheduleService.cs

```
using NaijaPrimeSchool.Application.Common;
```

```

using NaijaPrimeSchool.Application.Finance.Dtos;

namespace NaijaPrimeSchool.Application.Finance;

public interface IFeeScheduleService
{
    Task<IReadOnlyList<FeeScheduleDto>> ListAsync(FeeScheduleFilter filter,
CancellationToken ct = default);
    Task<FeeScheduleDetailDto?> GetByIdAsync(Guid id, CancellationToken ct = default);

    Task<OperationResult<Guid>> CreateAsync(CreateFeeScheduleRequest request,
CancellationToken ct = default);
    Task<OperationResult> UpdateAsync(UpdateFeeScheduleRequest request, CancellationToken
ct = default);
    Task<OperationResult> PublishAsync(Guid id, CancellationToken ct = default);
    Task<OperationResult> UnpublishAsync(Guid id, CancellationToken ct = default);
    Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct = default);

    Task<OperationResult<Guid>> UpsertItemAsync(UpsertFeeScheduleItemRequest request,
CancellationToken ct = default);
    Task<OperationResult> RemoveItemAsync(Guid itemId, CancellationToken ct = default);
}

```

Listing — src/NaijaPrimeSchool.Application/Finance/InvoiceService.cs

```

using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Finance.Dtos;

namespace NaijaPrimeSchool.Application.Finance;

public interface IInvoiceService
{
    Task<IReadOnlyList<InvoiceDto>> ListAsync(InvoiceFilter filter, CancellationToken ct =
default);
    Task<InvoiceDetailDto?> GetByIdAsync(Guid id, CancellationToken ct = default);

    Task<OperationResult<IssueInvoicesResponse>> IssueAsync(IssueInvoicesRequest request,
CancellationToken ct = default);
    Task<OperationResult> SetLineDiscountAsync(UpdateInvoiceLineDiscountRequest request,
CancellationToken ct = default);
    Task<OperationResult> CancelAsync(Guid id, CancellationToken ct = default);
    Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct = default);

    Task<StudentLedgerDto?> GetStudentLedgerAsync(Guid studentId, Guid? termId = null,
CancellationToken ct = default);
}

```

Listing — src/NaijaPrimeSchool.Application/Finance/IPaymentService.cs

```

using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Finance.Dtos;

namespace NaijaPrimeSchool.Application.Finance;

```



```

public interface IPaymentService
{
    Task<IReadOnlyList<PaymentDto>> ListAsync(PaymentFilter filter, CancellationToken ct = default);
    Task<PaymentDetailDto?> GetByIdAsync(Guid id, CancellationToken ct = default);

    Task<OperationResult<Guid>> RecordAsync(RecordPaymentRequest request, CancellationToken ct = default);
    Task<OperationResult> UpdateAsync(UpdatePaymentRequest request, CancellationToken ct = default);
    Task<OperationResult> RefundAsync(Guid id, CancellationToken ct = default);
    Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct = default);

    Task<FinanceSummaryDto> GetSummaryAsync(Guid? termId, CancellationToken ct = default);
}

```

4.6 ILookupService extension

Excerpt — ILookupService.cs (sprint 6 additions)

```

Task<IReadOnlyList<LookupDto>> GetFeeCategoriesAsync(CancellationToken ct = default);
Task<IReadOnlyList<LookupDto>> GetPaymentMethodsAsync(CancellationToken ct = default);
Task<IReadOnlyList<LookupDto>> GetInvoiceStatusesAsync(CancellationToken ct = default);
Task<IReadOnlyList<LookupDto>> GetPaymentStatusesAsync(CancellationToken ct = default);

```

5. Infrastructure — DbContext changes

5.1 Ten new DbSet

Excerpt — the ten finance DbSet

```
public DbSet<FeeCategory> FeeCategories => Set<FeeCategory>();
public DbSet<PaymentMethod> PaymentMethods => Set<PaymentMethod>();
public DbSet<InvoiceStatus> InvoiceStatuses => Set<InvoiceStatus>();
public DbSet<PaymentStatus> PaymentStatuses => Set<PaymentStatus>();
public DbSet<FeeSchedule> FeeSchedules => Set<FeeSchedule>();
public DbSet<FeeScheduleItem> FeeScheduleItems => Set<FeeScheduleItem>();
public DbSet<Invoice> Invoices => Set<Invoice>();
public DbSet<InvoiceLine> Invoicelines => Set<InvoiceLine>();
public DbSet<Payment> Payments => Set<Payment>();
public DbSet<PaymentAllocation> PaymentAllocations => Set<PaymentAllocation>();
```

5.2 ConfigureFinance

Excerpt — ConfigureFinance

```
private static void ConfigureFinance(ModelBuilder builder)
{
    ConfigureLookup<FeeCategory>(builder, "FeeCategories", extra: b =>
    {
        b.Property(c => c.Name).HasMaxLength(60).IsRequired();
        b.Property(c => c.Code).HasMaxLength(10).IsRequired();
        b.HasIndex(c => c.Name).IsUnique();
        b.HasIndex(c => c.Code).IsUnique();
    });

    ConfigureLookup<PaymentMethod>(builder, "PaymentMethods", extra: b =>
    {
        b.Property(m => m.Name).HasMaxLength(60).IsRequired();
        b.Property(m => m.Code).HasMaxLength(10).IsRequired();
        b.HasIndex(m => m.Name).IsUnique();
        b.HasIndex(m => m.Code).IsUnique();
    });

    ConfigureLookup<InvoiceStatus>(builder, "InvoiceStatuses", extra: b =>
    {
        b.Property(s => s.Name).HasMaxLength(40).IsRequired();
        b.Property(s => s.Code).HasMaxLength(20).IsRequired();
        b.HasIndex(s => s.Name).IsUnique();
        b.HasIndex(s => s.Code).IsUnique();
    });

    ConfigureLookup<PaymentStatus>(builder, "PaymentStatuses", extra: b =>
    {
        b.Property(s => s.Name).HasMaxLength(40).IsRequired();
        b.Property(s => s.Code).HasMaxLength(20).IsRequired();
        b.HasIndex(s => s.Name).IsUnique();
        b.HasIndex(s => s.Code).IsUnique();
    });
}
```

```

builder.Entity<FeeSchedule>(b =>
{
    b.ToTable("FeeSchedules");
    b.HasKey(s => s.Id);
    b.Property(s => s.Title).HasMaxLength(120).IsRequired();
    b.Property(s => s.Notes).HasMaxLength(500);
    b.Property(s => s.CreatedBy).HasMaxLength(100);
    b.Property(s => s.ModifiedBy).HasMaxLength(100);
    b.Property(s => s.DeletedBy).HasMaxLength(100);

    b.HasOne(s => s.Term).WithMany(t => t.FeeSchedules)
        .HasForeignKey(s => s.TermId)
        .OnDelete(DeleteBehavior.Restrict);

    b.HasOne(s => s.ClassLevel).WithMany(cl => cl.FeeSchedules)
        .HasForeignKey(s => s.ClassLevelId)
        .OnDelete(DeleteBehavior.Restrict);

    b.HasIndex(s => new { s.TermId, s.ClassLevelId }).IsUnique();
    b.HasIndex(s => s.IsDeleted);
    b.HasQueryFilter(s => !s.IsDeleted);
});

builder.Entity<FeeScheduleItem>(b =>
{
    b.ToTable("FeeScheduleItems");
    b.HasKey(i => i.Id);
    b.Property(i => i.Description).HasMaxLength(160).IsRequired();
    b.Property(i => i.Amount).HasPrecision(12, 2);
    b.Property(i => i.CreatedBy).HasMaxLength(100);
    b.Property(i => i.ModifiedBy).HasMaxLength(100);
    b.Property(i => i.DeletedBy).HasMaxLength(100);

    b.HasOne(i => i.FeeSchedule).WithMany(s => s.Items)
        .HasForeignKey(i => i.FeeScheduleId)
        .OnDelete(DeleteBehavior.Cascade);

    b.HasOne(i => i.FeeCategory).WithMany(c => c.ScheduleItems)
        .HasForeignKey(i => i.FeeCategoryId)
        .OnDelete(DeleteBehavior.Restrict);

    b.HasIndex(i => new { i.FeeScheduleId, i.DisplayOrder });
    b.HasIndex(i => i.IsDeleted);
    b.HasQueryFilter(i => !i.IsDeleted);
});

builder.Entity<Invoice>(b =>
{
    b.ToTable("Invoices");
    b.HasKey(i => i.Id);
    b.Property(i => i.InvoiceNumber).HasMaxLength(40).IsRequired();
    b.Property(i => i.Notes).HasMaxLength(500);
    b.Property(i => i.Subtotal).HasPrecision(12, 2);

```

```

        b.Property(i => i.DiscountTotal).HasPrecision(12, 2);
        b.Property(i => i.AmountDue).HasPrecision(12, 2);
        b.Property(i => i.AmountPaid).HasPrecision(12, 2);
        b.Property(i => i.CreatedBy).HasMaxLength(100);
        b.Property(i => i.ModifiedBy).HasMaxLength(100);
        b.Property(i => i.DeletedBy).HasMaxLength(100);
        b.Ignore(i => i.Balance);

        b.HasOne(i => i.Student).WithMany(s => s.Invoices)
            .HasForeignKey(i => i.StudentId)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasOne(i => i.Term).WithMany(t => t.Invoices)
            .HasForeignKey(i => i.TermId)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasOne(i => i.SchoolClass).WithMany(c => c.Invoices)
            .HasForeignKey(i => i.SchoolClassId)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasOne(i => i.InvoiceStatus).WithMany(s => s.Invoices)
            .HasForeignKey(i => i.InvoiceStatusId)
            .OnDelete(DeleteBehavior.Restrict);

        b.HasIndex(i => i.InvoiceNumber).IsUnique();
        b.HasIndex(i => new { i.StudentId, i.TermId });
        b.HasIndex(i => i.IsDeleted);
        b.HasQueryFilter(i => !i.IsDeleted);
    });

builder.Entity<InvoiceLine>(b =>
{
    b.ToTable("InvoiceLines");
    b.HasKey(l => l.Id);
    b.Property(l => l.Description).HasMaxLength(160).IsRequired();
    b.Property(l => l.Amount).HasPrecision(12, 2);
    b.Property(l => l.Discount).HasPrecision(12, 2);
    b.Property(l => l.CreatedBy).HasMaxLength(100);
    b.Property(l => l.ModifiedBy).HasMaxLength(100);
    b.Property(l => l.DeletedBy).HasMaxLength(100);
    b.Ignore(l => l.LineTotal);

    b.HasOne(l => l.Invoice).WithMany(i => i.Lines)
        .HasForeignKey(l => l.InvoiceId)
        .OnDelete(DeleteBehavior.Cascade);

    b.HasOne(l => l.FeeCategory).WithMany(c => c.InvoiceLines)
        .HasForeignKey(l => l.FeeCategoryId)
        .OnDelete(DeleteBehavior.Restrict);

    b.HasOne(l => l.FeeScheduleItem).WithMany()
        .HasForeignKey(l => l.FeeScheduleItemId)
        .OnDelete(DeleteBehavior.SetNull);
});

```

```

        b.HasIndex(l => l.IsDeleted);
        b.HasQueryFilter(l => !l.IsDeleted);
    });

builder.Entity<Payment>(b =>
{
    b.ToTable("Payments");
    b.HasKey(p => p.Id);
    b.Property(p => p.ReceiptNumber).HasMaxLength(40).IsRequired();
    b.Property(p => p.Amount).HasPrecision(12, 2);
    b.Property(p => p.Reference).HasMaxLength(120);
    b.Property(p => p.Notes).HasMaxLength(300);
    b.Property(p => p.CreatedBy).HasMaxLength(100);
    b.Property(p => p.ModifiedBy).HasMaxLength(100);
    b.Property(p => p.DeletedBy).HasMaxLength(100);

    b.HasOne(p => p.Student).WithMany(s => s.Payments)
        .HasForeignKey(p => p.StudentId)
        .OnDelete(DeleteBehavior.Restrict);

    b.HasOne(p => p.PaymentMethod).WithMany(m => m.Payments)
        .HasForeignKey(p => p.PaymentMethodId)
        .OnDelete(DeleteBehavior.Restrict);

    b.HasOne(p => p.PaymentStatus).WithMany(s => s.Payments)
        .HasForeignKey(p => p.PaymentStatusId)
        .OnDelete(DeleteBehavior.Restrict);

    b.HasOne(p => p.CollectedBy).WithMany()
        .HasForeignKey(p => p.CollectById)
        .OnDelete(DeleteBehavior.SetNull);

    b.HasIndex(p => p.ReceiptNumber).IsUnique();
    b.HasIndex(p => p.PaidOn);
    b.HasIndex(p => p.IsDeleted);
    b.HasQueryFilter(p => !p.IsDeleted);
});

builder.Entity<PaymentAllocation>(b =>
{
    b.ToTable("PaymentAllocations");
    b.HasKey(a => a.Id);
    b.Property(a => a.AmountApplied).HasPrecision(12, 2);
    b.Property(a => a.CreatedBy).HasMaxLength(100);
    b.Property(a => a.ModifiedBy).HasMaxLength(100);
    b.Property(a => a.DeletedBy).HasMaxLength(100);

    b.HasOne(a => a.Payment).WithMany(p => p.Allocations)
        .HasForeignKey(a => a.PaymentId)
        .OnDelete(DeleteBehavior.Cascade);

    b.HasOne(a => a.Invoice).WithMany(i => i.Allocations)
        .HasForeignKey(a => a.InvoiceId)
        .OnDelete(DeleteBehavior.Restrict);

```

```

        // One allocation row per (Payment, Invoice) pair.
        b.HasIndex(a => new { a.PaymentId, a.InvoiceId }).IsUnique();
        b.HasIndex(a => a.IsDeleted);
        b.HasQueryFilter(a => !a.IsDeleted);
    });
}

private static void ConfigureLookup<TEntity>(

```

Highlights:

- HasPrecision(12, 2) on every money column.
- Cascade delete on FeeScheduleItem.FeeScheduleId, InvoiceLine.InvoiceId, PaymentAllocation.PaymentId — the child rows always belong to their parent.
- Restrict on every cross-aggregate FK (StudentId, TermId, SchoolClassId, ClassLevelId, FeeCategoryId, InvoiceId on the allocation side, ...) so a schema-level constraint reinforces the soft-delete guards in the service layer.
- Unique indexes on Invoice.InvoiceNumber, Payment.ReceiptNumber, FeeSchedule (TermId, ClassLevelId), and PaymentAllocation (PaymentId, InvoiceId).
- b.Ignore(i => i.Balance) and b.Ignore(l => l.LineTotal) so EF Core does not try to materialise the computed properties as columns.

6. Infrastructure — service implementations

6.1 FeeScheduleService.cs

Listing — src/NaijaPrimeSchool.Infrastructure/Services/FeeScheduleService.cs

```
using Microsoft.EntityFrameworkCore;
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Finance;
using NaijaPrimeSchool.Application.Finance.Dtos;
using NaijaPrimeSchool.Domain.Finance;
using NaijaPrimeSchool.Infrastructure.Persistence;

namespace NaijaPrimeSchool.Infrastructure.Services;

public class FeeScheduleService(ApplicationDbContext db) : IFeeScheduleService
{
    private IQueryable<FeeScheduleDto> Project(IQueryable<FeeSchedule> q) =>
        q.Select(s => new FeeScheduleDto
        {
            Id = s.Id,
            TermId = s.TermId,
            TermName = s.Term!.TermType!.Name + " - " + s.Term!.Session!.Name,
            SessionId = s.Term!.SessionId,
            SessionName = s.Term!.Session!.Name,
            ClassLevelId = s.ClassLevelId,
            ClassLevelName = s.ClassLevel!.Name,
            Title = s.Title,
            Notes = s.Notes,
            IsPublished = s.IsPublished,
            PublishedOn = s.PublishedOn,
            ItemCount = s.Items.Count,
            TotalAmount = s.Items.Sum(i => (decimal?)i.Amount) ?? 0m,
            InvoicesIssued = db.InvoiceLines
                .Count(l => l.FeeScheduleItem != null
                    && l.FeeScheduleItem.FeeScheduleId == s.Id),
        });

    public async Task<IReadOnlyList<FeeScheduleDto>> ListAsync(FeeScheduleFilter filter,
        CancellationToken ct = default)
    {
        var q = db.FeeSchedules.AsQueryable();
        if (filter.TermId.HasValue) q = q.Where(s => s.TermId == filter.TermId.Value);
        if (filter.ClassLevelId.HasValue) q = q.Where(s => s.ClassLevelId ==
filter.ClassLevelId.Value);
        if (filter.IsPublished.HasValue) q = q.Where(s => s.IsPublished ==
filter.IsPublished.Value);

        return await Project(q
            .OrderBy(s => s.ClassLevel!.DisplayOrder)
            .ThenBy(s => s.Title))
            .ToListAsync(ct);
    }

    public async Task<FeeScheduleDetailDto?> GetByIdAsync(Guid id, CancellationToken ct =
```

```

default)
{
    var schedule = await Project(db.FeeSchedules.Where(s => s.Id == id))
        .FirstOrDefaultAsync(ct);
    if (schedule is null) return null;

    var items = await db.FeeScheduleItems
        .Where(i => i.FeeScheduleId == id)
        .OrderBy(i => i.DisplayOrder)
        .ThenBy(i => i.FeeCategory!.DisplayOrder)
        .Select(i => new FeeScheduleItemDto
        {
            Id = i.Id,
            FeeScheduleId = i.FeeScheduleId,
            FeeCategoryId = i.FeeCategoryId,
            FeeCategoryName = i.FeeCategory!.Name,
            FeeCategoryCode = i.FeeCategory!.Code,
            Description = i.Description,
            Amount = i.Amount,
            IsMandatory = i.IsMandatory,
            DisplayOrder = i.DisplayOrder,
        })
        .ToListAsync(ct);

    return new FeeScheduleDetailDto { Schedule = schedule, Items = items };
}

public async Task<OperationResult<Guid>> CreateAsync(CreateFeeScheduleRequest request,
    CancellationToken ct = default)
{
    if (!await db.Terms.AnyAsync(t => t.Id == request.TermId, ct))
        return OperationResult<Guid>.Failure("Term not found.");

    if (!await db.ClassLevels.AnyAsync(l => l.Id == request.ClassLevelId, ct))
        return OperationResult<Guid>.Failure("Class level not found.");

    if (await db.FeeSchedules.AnyAsync(s =>
        s.TermId == request.TermId && s.ClassLevelId == request.ClassLevelId, ct))
        return OperationResult<Guid>.Failure(
            "A fee schedule already exists for this (term, class level). Edit that one
instead.");

    var schedule = new FeeSchedule
    {
        TermId = request.TermId,
        ClassLevelId = request.ClassLevelId,
        Title = request.Title.Trim(),
        Notes = request.Notes,
    };
    db.FeeSchedules.Add(schedule);
    await db.SaveChangesAsync(ct);
    return OperationResult<Guid>.Success(schedule.Id);
}

public async Task<OperationResult> UpdateAsync(UpdateFeeScheduleRequest request,

```



```

CancellationToken ct = default)
{
    var schedule = await db.FeeSchedules.FirstOrDefaultAsync(s => s.Id == request.Id,
ct);
    if (schedule is null) return OperationResult.Failure("Fee schedule not found.");
    if (schedule.IsPublished) return OperationResult.Failure(
        "Unpublish the fee schedule before editing.");

    schedule.Title = request.Title.Trim();
    schedule.Notes = request.Notes;
    await db.SaveChangesAsync(ct);
    return OperationResult.Success();
}

public async Task<OperationResult> PublishAsync(Guid id, Cancellation_token ct =
default)
{
    var schedule = await db.FeeSchedules
        .Include(s => s.Items)
        .FirstOrDefaultAsync(s => s.Id == id, ct);
    if (schedule is null) return OperationResult.Failure("Fee schedule not found.");
    if (schedule.Items.Count == 0)
        return OperationResult.Failure(
            "Add at least one line item before publishing the schedule.");

    schedule.IsPublished = true;
    schedule.PublishedOn = DateTimeOffset.UtcNow;
    await db.SaveChangesAsync(ct);
    return OperationResult.Success();
}

public async Task<OperationResult> UnpublishAsync(Guid id, CancellationToken ct =
default)
{
    var schedule = await db.FeeSchedules.FirstOrDefaultAsync(s => s.Id == id, ct);
    if (schedule is null) return OperationResult.Failure("Fee schedule not found.");

    var hasInvoices = await db.InvoiceLines
        .AnyAsync(l => l.FeeScheduleItem != null
            && l.FeeScheduleItem.FeeScheduleId == id, ct);
    if (hasInvoices)
        return OperationResult.Failure(
            "Invoices have already been issued from this schedule. Cancel the affected
invoices first.");

    schedule.IsPublished = false;
    schedule.PublishedOn = null;
    await db.SaveChangesAsync(ct);
    return OperationResult.Success();
}

public async Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct =
default)
{

```

```

        var schedule = await db.FeeSchedules
            .Include(s => s.Items)
            .FirstOrDefaultAsync(s => s.Id == id, ct);
        if (schedule is null) return OperationResult.Failure("Fee schedule not found.");

        var hasInvoices = await db.InvoiceLines
            .AnyAsync(l => l.FeeScheduleItem != null
                && l.FeeScheduleItem.FeeScheduleId == id, ct);
        if (hasInvoices)
            return OperationResult.Failure(
                "Cannot delete a schedule that has been used to issue invoices.");

        db.FeeSchedules.Remove(schedule);
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult<Guid>> UpsertItemAsync(UpsertFeeScheduleItemRequest
        request, CancellationToken ct = default)
    {
        var schedule = await db.FeeSchedules.FirstOrDefaultAsync(s => s.Id ==
            request.FeeScheduleId, ct);
        if (schedule is null)
            return OperationResult<Guid>.Failure("Fee schedule not found.");
        if (schedule.IsPublished)
            return OperationResult<Guid>.Failure("Unpublish the schedule before editing
                items.");

        if (!await db.FeeCategories.AnyAsync(c => c.Id == request.FeeCategoryId, ct))
            return OperationResult<Guid>.Failure("Fee category not found.");

        if (request.Amount <= 0)
            return OperationResult<Guid>.Failure("Amount must be greater than zero.");

        FeeScheduleItem item;
        if (request.Id.HasValue)
        {
            item = await db.FeeScheduleItems.FirstOrDefaultAsync(i => i.Id ==
                request.Id.Value, ct)
                ?? throw new InvalidOperationException("Item not found.");
            item.FeeCategoryId = request.FeeCategoryId;
            item.Description = request.Description.Trim();
            item.Amount = request.Amount;
            item.IsMandatory = request.IsMandatory;
            item.DisplayOrder = request.DisplayOrder;
        }
        else
        {
            item = new FeeScheduleItem
            {
                FeeScheduleId = request.FeeScheduleId,
                FeeCategoryId = request.FeeCategoryId,
                Description = request.Description.Trim(),
                Amount = request.Amount,
            }
        }
    }

```

```

        IsMandatory = request.IsMandatory,
        DisplayOrder = request.DisplayOrder,
    };
    db.FeeScheduleItems.Add(item);
}

await db.SaveChangesAsync(ct);
return OperationResult<Guid>.Success(item.Id);
}

public async Task<OperationResult> RemoveItemAsync(Guid itemId, CancellationToken ct =
default)
{
    var item = await db.FeeScheduleItems
        .Include(i => i.FeeSchedule)
        .FirstOrDefaultAsync(i => i.Id == itemId, ct);
    if (item is null) return OperationResult.Failure("Item not found.");
    if (item.FeeSchedule!.IsPublished)
        return OperationResult.Failure("Unpublish the schedule before removing
items.");

    db.FeeScheduleItems.Remove(item);
    await db.SaveChangesAsync(ct);
    return OperationResult.Success();
}
}

```

6.2 InvoiceService.cs

InvoiceService owns the schedule-to-invoice fan-out, per-line discount application, cancellation, and the per-student ledger. The internal helper `RecomputeInvoiceTotalsAsync` is shared with `PaymentService` via the concrete-class injection described in chapter 2.

Listing — src/NaijaPrimeSchool.Infrastructure/Services/InvoiceService.cs

```

using Microsoft.EntityFrameworkCore;
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Finance;
using NaijaPrimeSchool.Application.Finance.Dtos;
using NaijaPrimeSchool.Domain.Finance;
using NaijaPrimeSchool.Infrastructure.Persistence;

namespace NaijaPrimeSchool.Infrastructure.Services;

public class InvoiceService(ApplicationDbContext db) : IInvoiceService
{
    private static IQueryable<InvoiceDto> Project(IQueryable<Invoice> q) =>
        q.Select(i => new InvoiceDto
        {
            Id = i.Id,
            InvoiceNumber = i.InvoiceNumber,
            IssuedOn = i.IssuedOn,
            DueDate = i.DueDate,
            StudentId = i.StudentId,

```

```

        StudentName = (i.Student!.FirstName + " " + i.Student!.LastName).Trim(),
        StudentAdmissionNumber = i.Student!.AdmissionNumber,
        StudentPhotoUrl = i.Student!.PhotoUrl,
        StudentFirstName = i.Student!.FirstName,
        StudentLastName = i.Student!.LastName,
        TermId = i.TermId,
        TermName = i.Term!.TermType!.Name + " - " + i.Term!.Session!.Name,
        SessionId = i.Term!.SessionId,
        SessionName = i.Term!.Session!.Name,
        SchoolClassId = i.SchoolClassId,
        SchoolClassName = i.SchoolClass!.Name,
        InvoiceStatusId = i.InvoiceStatusId,
        InvoiceStatusName = i.InvoiceStatus!.Name,
        InvoiceStatusCode = i.InvoiceStatus!.Code,
        Subtotal = i.Subtotal,
        DiscountTotal = i.DiscountTotal,
        AmountDue = i.AmountDue,
        AmountPaid = i.AmountPaid,
        Notes = i.Notes,
    });

    public async Task<IReadOnlyList<InvoiceDto>> ListAsync(InvoiceFilter filter,
        CancellationToken ct = default)
    {
        var q = db.Invoices.AsQueryable();
        if (filter.StudentId.HasValue) q = q.Where(i => i.StudentId ==
filter.StudentId.Value);
        if (filter.TermId.HasValue) q = q.Where(i => i.TermId == filter.TermId.Value);
        if (filter.SessionId.HasValue) q = q.Where(i => i.Term!.SessionId ==
filter.SessionId.Value);
        if (filter.SchoolClassId.HasValue) q = q.Where(i => i.SchoolClassId ==
filter.SchoolClassId.Value);
        if (filter.InvoiceStatusId.HasValue) q = q.Where(i => i.InvoiceStatusId ==
filter.InvoiceStatusId.Value);
        if (!string.IsNullOrEmpty(filter.Search))
        {
            var term = filter.Search.Trim().ToLower();
            q = q.Where(i =>
                i.InvoiceNumber.ToLower().Contains(term)
                || i.Student!.FirstName.ToLower().Contains(term)
                || i.Student!.LastName.ToLower().Contains(term)
                || i.Student!.AdmissionNumber.ToLower().Contains(term));
        }

        return await Project(q.OrderByDescending(i => i.IssuedOn).ThenBy(i =>
i.InvoiceNumber))
            .ToListAsync(ct);
    }

    public async Task<InvoiceDetailDto?> GetByIdAsync(Guid id, CancellationToken ct =
default)
    {
        var invoice = await Project(db.Invoices.Where(i => i.Id == id))
            .FirstOrDefaultAsync(ct);
        if (invoice is null) return null;
    }

```

```

var lines = await db.InvoiceLines
    .Where(l => l.InvoiceId == id)
    .OrderBy(l => l.FeeCategory!.DisplayOrder)
    .Select(l => new InvoiceLineDto
    {
        Id = l.Id,
        InvoiceId = l.InvoiceId,
        FeeCategoryId = l.FeeCategoryId,
        FeeCategoryName = l.FeeCategory!.Name,
        Description = l.Description,
        Amount = l.Amount,
        Discount = l.Discount,
    })
    .ToListAsync(ct);

var payments = await db.PaymentAllocations
    .Where(a => a.InvoiceId == id)
    .OrderByDescending(a => a.Payment!.PaidOn)
    .Select(a => new InvoicePaymentDto
    {
        PaymentId = a.PaymentId,
        ReceiptNumber = a.Payment!.ReceiptNumber,
        PaidOn = a.Payment!.PaidOn,
        PaymentMethodName = a.Payment!.PaymentMethod!.Name,
        AmountApplied = a.AmountApplied,
        Reference = a.Payment!.Reference,
    })
    .ToListAsync(ct);

return new InvoiceDetailDto { Invoice = invoice, Lines = lines, Payments = payments
};
}

public async Task<OperationResult<IssueInvoicesResponse>>
IssueAsync(IssueInvoicesRequest request, CancellationToken ct = default)
{
    var schedule = await db.FeeSchedules
        .Include(s => s.Items)
        .ThenInclude(i => i.FeeCategory)
        .Include(s => s.Term)
        .FirstOrDefaultAsync(s => s.Id == request.FeeScheduleId, ct);
    if (schedule is null)
        return OperationResult<IssueInvoicesResponse>.Failure("Fee schedule not
found.");
    if (!schedule.IsPublished)
        return OperationResult<IssueInvoicesResponse>.Failure(
            "Publish the fee schedule before issuing invoices from it.");
    if (schedule.Items.Count == 0)
        return OperationResult<IssueInvoicesResponse>.Failure(
            "The fee schedule has no line items.");

    var schoolClass = await db.SchoolClasses.FirstOrDefaultAsync(c => c.Id ==
request.SchoolClassId, ct);

```

```

if (schoolClass is null)
    return OperationResult<IssueInvoicesResponse>.Failure("Class not found.");

if (schoolClass.ClassLevelId != schedule.ClassLevelId)
    return OperationResult<IssueInvoicesResponse>.Failure(
        "Class belongs to a different class level than the schedule.");

var enrolledStudents = await db.Enrolments
    .Where(e => e.SchoolClassId == request.SchoolClassId && e.WithdrawnOn == null)
    .Select(e => e.StudentId)
    .Distinct()
    .ToListAsync(ct);

if (request.StudentIds is { Count: > 0 })
    enrolledStudents = enrolledStudents.Intersect(request.StudentIds).ToList();

if (enrolledStudents.Count == 0)
    return OperationResult<IssueInvoicesResponse>.Failure(
        "No actively-enrolled pupils in the selected class.");

var existing = await db.Invoices
    .Where(i => i.TermId == schedule.TermId
        && i.SchoolClassId == request.SchoolClassId
        && enrolledStudents.Contains(i.StudentId))
    .Select(i => i.StudentId)
    .ToListAsync(ct);

var issuedStatusId = await db.InvoiceStatuses
    .Where(s => s.Code == "ISSUED")
    .Select(s => s.Id)
    .FirstOrDefaultAsync(ct);
if (issuedStatusId == Guid.Empty)
    return OperationResult<IssueInvoicesResponse>.Failure(
        "Invoice statuses are not seeded.");

// Compute a per-issuance starting sequence by counting existing invoices
// for the issue year.
var year = request.IssuedOn.Year;
var yearPrefix = $"NPS/INV/{year}/";
var maxSeq = await db.Invoices
    .Where(i => i.InvoiceNumber.StartsWith(yearPrefix))
    .Select(i => i.InvoiceNumber)
    .ToListAsync(ct);
int nextSeq = ParseHighestSequence(maxSeq, yearPrefix) + 1;

var issued = 0;
var skipped = 0;
var warnings = new List<string>();
var subtotal = schedule.Items.Sum(i => i.Amount);

foreach (var studentId in enrolledStudents)
{
    if (existing.Contains(studentId))
    {

```

```

        skipped++;
        continue;
    }

    var invoice = new Invoice
    {
        StudentId = studentId,
        TermId = schedule.TermId,
        SchoolClassId = request.SchoolClassId,
        InvoiceStatusId = issuedStatusId,
        InvoiceNumber = yearPrefix + nextSeq.ToString("D4"),
        IssuedOn = request.IssuedOn,
        DueDate = request.DueDate,
        Subtotal = subtotal,
        DiscountTotal = 0m,
        AmountDue = subtotal,
        AmountPaid = 0m,
    };
    foreach (var item in schedule.Items)
    {
        invoice.Lines.Add(new InvoiceLine
        {
            FeeCategoryId = item.FeeCategoryId,
            FeeScheduleItemId = item.Id,
            Description = item.Description,
            Amount = item.Amount,
            Discount = 0m,
        });
    }
    db.Invoices.Add(invoice);
    issued++;
    nextSeq++;
}

if (skipped > 0)
    warnings.Add($"{skipped} pupil(s) already had an invoice for this term/class.
Skipped.");

await db.SaveChangesAsync(ct);
return OperationResult<IssueInvoicesResponse>.Success(new IssueInvoicesResponse
{
    Issued = issued,
    Skipped = skipped,
    Warnings = warnings,
});
}

private static int ParseHighestSequence(IEnumerable<string> numbers, string prefix)
{
    var max = 0;
    foreach (var n in numbers)
    {
        if (!n.StartsWith(prefix)) continue;
        if (int.TryParse(n[prefix.Length..], out var seq) && seq > max) max = seq;
    }
}

```

```

    }
    return max;
}

public async Task<OperationResult>
SetLineDiscountAsync(UpdateInvoiceLineDiscountRequest request, CancellationToken ct =
default)
{
    var line = await db.InvoiceLines
        .Include(l => l.Invoice)
        .FirstOrDefaultAsync(l => l.Id == request.InvoiceLineId, ct);
    if (line is null) return OperationResult.Failure("Invoice line not found.");

    if (request.Discount < 0 || request.Discount > line.Amount)
        return OperationResult.Failure(
            $"Discount must be between 0 and {line.Amount:N2}.");

    var invoice = line.Invoice!;
    if (invoice.InvoiceStatus is null)
        invoice.InvoiceStatus = await db.InvoiceStatuses
            .FirstOrDefaultAsync(s => s.Id == invoice.InvoiceStatusId, ct);

    if (invoice.InvoiceStatus?.Code == "CANCELLED")
        return OperationResult.Failure("Cancelled invoices cannot be edited.");

    line.Discount = request.Discount;

    // Recompute totals from the lines.
    await RecomputeInvoiceTotalsAsync(invoice.Id, ct);
    await db.SaveChangesAsync(ct);
    return OperationResult.Success();
}

public async Task<OperationResult> CancelAsync(Guid id, CancellationToken ct = default)
{
    var invoice = await db.Invoices
        .Include(i => i.Allocations)
        .FirstOrDefaultAsync(i => i.Id == id, ct);
    if (invoice is null) return OperationResult.Failure("Invoice not found.");

    if (invoice.Allocations.Any(a => a.AmountApplied > 0))
        return OperationResult.Failure(
            "Cannot cancel an invoice that has payments applied. Refund the payments
first.");

    var cancelledId = await db.InvoiceStatuses
        .Where(s => s.Code == "CANCELLED")
        .Select(s => s.Id)
        .FirstOrDefaultAsync(ct);
    if (cancelledId == Guid.Empty)
        return OperationResult.Failure("Cancelled status is not seeded.");

    invoice.InvoiceStatusId = cancelledId;
    await db.SaveChangesAsync(ct);
}

```



```

        return OperationResult.Success();
    }

    public async Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct =
default)
    {
        var invoice = await db.Invoices
            .Include(i => i.Allocations)
            .FirstOrDefaultAsync(i => i.Id == id, ct);
        if (invoice is null) return OperationResult.Failure("Invoice not found.");

        if (invoice.Allocations.Any(a => a.AmountApplied > 0))
            return OperationResult.Failure(
                "Cannot delete an invoice with payments applied.");

        db.Invoices.Remove(invoice);
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<StudentLedgerDto?> GetStudentLedgerAsync(Guid studentId, Guid? termId
= null, CancellationToken ct = default)
    {
        var student = await db.Students
            .Where(s => s.Id == studentId)
            .Select(s => new
            {
                s.Id,
                Name = (s.FirstName + " " + s.LastName).Trim(),
                s.AdmissionNumber,
                s.PhotoUrl,
                s.FirstName,
                s.LastName,
            })
            .FirstOrDefaultAsync(ct);
        if (student is null) return null;

        var invoiceQuery = db.Invoices.Where(i => i.StudentId == studentId);
        if (termId.HasValue) invoiceQuery = invoiceQuery.Where(i => i.TermId ==
termId.Value);

        var invoices = await Project(invoiceQuery.OrderByDescending(i => i.IssuedOn))
            .ToListAsync(ct);

        var paymentQuery = db.Payments.Where(p => p.StudentId == studentId);
        if (termId.HasValue)
        {
            // Restrict payments to those that touched invoices in the term.
            paymentQuery = paymentQuery.Where(p =>
                p.Allocations.Any(a => a.Invoice!.TermId == termId.Value));
        }
        var payments = await paymentQuery
            .OrderByDescending(p => p.PaidOn)
            .Select(p => new PaymentDto

```

```

        {
            Id = p.Id,
            ReceiptNumber = p.ReceiptNumber,
            PaidOn = p.PaidOn,
            Amount = p.Amount,
            StudentId = p.StudentId,
            StudentName = student.Name,
            StudentAdmissionNumber = student.AdmissionNumber,
            StudentPhotoUrl = student.PhotoUrl,
            StudentFirstName = student.FirstName,
            StudentLastName = student.LastName,
            PaymentMethodId = p.PaymentMethodId,
            PaymentMethodName = p.PaymentMethod!.Name,
            PaymentStatusId = p.PaymentStatusId,
            PaymentStatusName = p.PaymentStatus!.Name,
            PaymentStatusCode = p.PaymentStatus!.Code,
            Reference = p.Reference,
            Notes = p.Notes,
            CollectedById = p.CollectedById,
            CollectedByName = p.CollectedBy == null
                ? null
                : (p.CollectedBy!.FirstName + " " + p.CollectedBy!.LastName).Trim(),
            AmountAllocated = p.Allocations.Sum(a => (decimal?)a.AmountApplied) ?? 0m,
        })
        .ToListAsync(ct);

return new StudentLedgerDto
{
    StudentId = studentId,
    StudentName = student.Name,
    StudentAdmissionNumber = student.AdmissionNumber,
    StudentPhotoUrl = student.PhotoUrl,
    StudentFirstName = student.FirstName,
    StudentLastName = student.LastName,
    TotalInvoiced = invoices.Sum(i => i.AmountDue),
    TotalPaid = invoices.Sum(i => i.AmountPaid),
    Invoices = invoices.ToList(),
    Payments = payments,
};
}

// Internal helper – also used by PaymentService after allocations.
internal async Task RecomputeInvoiceTotalsAsync(Guid invoiceId, CancellationToken ct)
{
    var invoice = await db.Invoices
        .Include(i => i.Lines)
        .Include(i => i.Allocations)
        .FirstOrDefaultAsync(i => i.Id == invoiceId, ct);
    if (invoice is null) return;

    invoice.Subtotal = invoice.Lines.Sum(l => l.Amount);
    invoice.DiscountTotal = invoice.Lines.Sum(l => l.Discount);
    invoice.AmountDue = invoice.Subtotal - invoice.DiscountTotal;
    invoice.AmountPaid = invoice.Allocations.Sum(a => a.AmountApplied);
}

```

```

        // Bring status into line with money flow.
        var statusCode = invoice.AmountPaid <= 0m
            ? "ISSUED"
            : invoice.AmountPaid >= invoice.AmountDue
            ? "PAID"
            : "PARTIAL";

        // Never overwrite a CANCELLED row from here.
        var currentStatus = await db.InvoiceStatuses
            .FirstOrDefaultAsync(s => s.Id == invoice.InvoiceStatusId, ct);
        if (currentStatus?.Code == "CANCELLED") return;

        var targetStatus = await db.InvoiceStatuses
            .FirstOrDefaultAsync(s => s.Code == statusCode, ct);
        if (targetStatus is not null) invoice.InvoiceStatusId = targetStatus.Id;
    }
}

```

6.3 PaymentService.cs

Listing — src/NaijaPrimeSchool.Infrastructure/Services/PaymentService.cs

```

using Microsoft.EntityFrameworkCore;
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Finance;
using NaijaPrimeSchool.Application.Finance.Dtos;
using NaijaPrimeSchool.Domain.Finance;
using NaijaPrimeSchool.Infrastructure.Persistence;

namespace NaijaPrimeSchool.Infrastructure.Services;

public class PaymentService(
    ApplicationDbContext db,
    InvoiceService invoiceService) : IPaymentService
{
    private static IQueryable<PaymentDto> Project(IQueryable<Payment> q) =>
        q.Select(p => new PaymentDto
        {
            Id = p.Id,
            ReceiptNumber = p.ReceiptNumber,
            PaidOn = p.PaidOn,
            Amount = p.Amount,
            StudentId = p.StudentId,
            StudentName = (p.Student!.FirstName + " " + p.Student!.LastName).Trim(),
            StudentAdmissionNumber = p.Student!.AdmissionNumber,
            StudentPhotoUrl = p.Student!.PhotoUrl,
            StudentFirstName = p.Student!.FirstName,
            StudentLastName = p.Student!.LastName,
            PaymentMethodId = p.PaymentMethodId,
            PaymentMethodName = p.PaymentMethod!.Name,
            PaymentStatusId = p.PaymentStatusId,
            PaymentStatusName = p.PaymentStatus!.Name,
            PaymentStatusCode = p.PaymentStatus!.Code,
            Reference = p.Reference,

```

```

        Notes = p.Notes,
        CollectedById = p.CollectedById,
        CollectedByName = p.CollectedBy == null
            ? null
            : (p.CollectedBy!.FirstName + " " + p.CollectedBy!.LastName).Trim(),
        AmountAllocated = p.Allocations.Sum(a => (decimal?)a.AmountApplied) ?? 0m,
    });

    public async Task<IReadOnlyList<PaymentDto>> ListAsync(PaymentFilter filter,
        CancellationToken ct = default)
    {
        var q = db.Payments.AsQueryable();
        if (filter.StudentId.HasValue) q = q.Where(p => p.StudentId ==
filter.StudentId.Value);
        if (filter.PaymentMethodId.HasValue) q = q.Where(p => p.PaymentMethodId ==
filter.PaymentMethodId.Value);
        if (filter.PaymentStatusId.HasValue) q = q.Where(p => p.PaymentStatusId ==
filter.PaymentStatusId.Value);
        if (filter.FromDate.HasValue) q = q.Where(p => p.PaidOn >= filter.FromDate.Value);
        if (filter.ToDate.HasValue) q = q.Where(p => p.PaidOn <= filter.ToDate.Value);
        if (!string.IsNullOrEmpty(filter.Search))
        {
            var term = filter.Search.Trim().ToLower();
            q = q.Where(p =>
                p.ReceiptNumber.ToLower().Contains(term)
                || (p.Reference != null && p.Reference.ToLower().Contains(term))
                || p.Student!.FirstName.ToLower().Contains(term)
                || p.Student!.LastName.ToLower().Contains(term)
                || p.Student!.AdmissionNumber.ToLower().Contains(term));
        }

        return await Project(q.OrderByDescending(p => p.PaidOn).ThenByDescending(p =>
p.ReceiptNumber))
            .ToListAsync(ct);
    }

    public async Task<PaymentDetailDto?> GetByIdAsync(Guid id, CancellationToken ct =
default)
    {
        var payment = await Project(db.Payments.Where(p => p.Id == id))
            .FirstOrDefaultAsync(ct);
        if (payment is null) return null;

        var allocations = await db.PaymentAllocations
            .Where(a => a.PaymentId == id)
            .OrderBy(a => a.Invoice!.IssuedOn)
            .Select(a => new PaymentAllocationDto
            {
                Id = a.Id,
                PaymentId = a.PaymentId,
                InvoiceId = a.InvoiceId,
                InvoiceNumber = a.Invoice!.InvoiceNumber,
                AmountApplied = a.AmountApplied,
                InvoiceBalanceAfter = a.Invoice!.AmountDue - a.Invoice!.AmountPaid,
            })
            .ToListAsync(ct);
    }

```

```

    })
    .ToListAsync(ct);

    return new PaymentDetailDto { Payment = payment, Allocations = allocations };
}

public async Task<OperationResult<Guid>> RecordAsync(RecordPaymentRequest request,
CancellationToken ct = default)
{
    if (request.Amount <= 0)
        return OperationResult<Guid>.Failure("Payment amount must be greater than
zero.");

    if (!await db.Students.AnyAsync(s => s.Id == request.StudentId, ct))
        return OperationResult<Guid>.Failure("Student not found.");
    if (!await db.PaymentMethods.AnyAsync(m => m.Id == request.PaymentMethodId, ct))
        return OperationResult<Guid>.Failure("Payment method not found.");

    var confirmedStatusId = await db.PaymentStatuses
        .Where(s => s.Code == "CONFIRMED")
        .Select(s => s.Id)
        .FirstOrDefaultAsync(ct);
    if (confirmedStatusId == Guid.Empty)
        return OperationResult<Guid>.Failure("Payment statuses are not seeded.");

    // Validate allocations against the student's invoices.
    var allocations = request.Allocations
        .Where(a => a.AmountApplied > 0m)
        .ToListAsync();
    var totalAllocated = allocations.Sum(a => a.AmountApplied);
    if (totalAllocated > request.Amount)
        return OperationResult<Guid>.Failure(
            $"Allocations ({totalAllocated:N2}) exceed payment amount
({request.Amount:N2}).");

    var invoiceIds = allocations.Select(a => a.InvoiceId).ToListAsync();
    var invoices = await db.Invoices
        .Where(i => invoiceIds.Contains(i.Id))
        .Include(i => i.InvoiceStatus)
        .ToListAsync(ct);

    if (invoices.Count != invoiceIds.Distinct().Count())
        return OperationResult<Guid>.Failure("One or more invoices were not found.");

    foreach (var allocation in allocations)
    {
        var invoice = invoices.First(i => i.Id == allocation.InvoiceId);
        if (invoice.StudentId != request.StudentId)
            return OperationResult<Guid>.Failure(
                $"Invoice {invoice.InvoiceNumber} does not belong to the selected
pupil.");
        if (invoice.InvoiceStatus?.Code == "CANCELLED")
            return OperationResult<Guid>.Failure(
                $"Invoice {invoice.InvoiceNumber} is cancelled – cannot accept

```

```

payment.");

        var outstanding = invoice.AmountDue - invoice.AmountPaid;
        if (allocation.AmountApplied > outstanding)
            return OperationResult<Guid>.Failure(
                $"Allocation for {invoice.InvoiceNumber}
({allocation.AmountApplied:N2}) "
                + $"exceeds the outstanding balance ({outstanding:N2}).");
    }

    // Receipt number is NPS/RCP/<year>/<sequence>.
    var year = request.PaidOn.Year;
    var prefix = $"NPS/RCP/{year}/";
    var existingNumbers = await db.Payments
        .Where(p => p.ReceiptNumber.StartsWith(prefix))
        .Select(p => p.ReceiptNumber)
        .ToListAsync(ct);
    var nextSeq = ParseHighestSequence(existingNumbers, prefix) + 1;

    var payment = new Payment
    {
        StudentId = request.StudentId,
        PaymentMethodId = request.PaymentMethodId,
        PaymentStatusId = confirmedStatusId,
        ReceiptNumber = prefix + nextSeq.ToString("D4"),
        PaidOn = request.PaidOn,
        Amount = request.Amount,
        Reference = request.Reference,
        Notes = request.Notes,
        CollectedById = request.CollectedById,
    };

    foreach (var a in allocations)
    {
        payment.Allocations.Add(new PaymentAllocation
        {
            InvoiceId = a.InvoiceId,
            AmountApplied = a.AmountApplied,
        });
    }
    db.Payments.Add(payment);
    await db.SaveChangesAsync(ct);

    foreach (var invoiceId in invoiceIds.Distinct())
    {
        await invoiceService.RecomputeInvoiceTotalsAsync(invoiceId, ct);
    }
    await db.SaveChangesAsync(ct);

    return OperationResult<Guid>.Success(payment.Id);
}

private static int ParseHighestSequence(IEnumerable<string> numbers, string prefix)
{

```

```

        var max = 0;
        foreach (var n in numbers)
        {
            if (!n.StartsWith(prefix)) continue;
            if (int.TryParse(n[prefix.Length..], out var seq) && seq > max) max = seq;
        }
        return max;
    }

    public async Task<OperationResult> UpdateAsync(UpdatePaymentRequest request,
CancellationTokens ct = default)
    {
        var payment = await db.Payments.FirstOrDefaultAsync(p => p.Id == request.Id, ct);
        if (payment is null) return OperationResult.Failure("Payment not found.");

        payment.Reference = request.Reference;
        payment.Notes = request.Notes;
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> RefundAsync(Guid id, CancellationTokens ct = default)
    {
        var payment = await db.Payments
            .Include(p => p.Allocations)
            .FirstOrDefaultAsync(p => p.Id == id, ct);
        if (payment is null) return OperationResult.Failure("Payment not found.");

        var refundedId = await db.PaymentStatuses
            .Where(s => s.Code == "REFUNDED")
            .Select(s => s.Id)
            .FirstOrDefaultAsync(ct);
        if (refundedId == Guid.Empty)
            return OperationResult.Failure("Refunded status is not seeded.");

        var touchedInvoices = payment.Allocations.Select(a =>
a.InvoiceId).Distinct().ToList();

        // Drop the allocations so the corresponding invoices no longer count this
        // payment toward their AmountPaid. We do not delete the Payment row –
        // the receipt history stays intact.
        db.PaymentAllocations.RemoveRange(payment.Allocations);
        payment.PaymentStatusId = refundedId;
        await db.SaveChangesAsync(ct);

        foreach (var invoiceId in touchedInvoices)
        {
            await invoiceService.RecomputeInvoiceTotalsAsync(invoiceId, ct);
        }
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> SoftDeleteAsync(Guid id, CancellationTokens ct =

```

```

default)
{
    var payment = await db.Payments
        .Include(p => p.Allocations)
        .FirstOrDefaultAsync(p => p.Id == id, ct);
    if (payment is null) return OperationResult.Failure("Payment not found.");

    if (payment.Allocations.Any())
        return OperationResult.Failure(
            "Refund the payment first so its allocations are released, then delete.");

    db.Payments.Remove(payment);
    await db.SaveChangesAsync(ct);
    return OperationResult.Success();
}

public async Task<FinanceSummaryDto> GetSummaryAsync(Guid? termId, CancellationToken ct
= default)
{
    var termName = termId.HasValue
        ? await db.Terms
            .Where(t => t.Id == termId.Value)
            .Select(t => t.TermType!.Name + " - " + t.Session!.Name)
            .FirstOrDefaultAsync(ct)
        : null;

    var invQuery = db.Invoices.AsQueryable();
    if (termId.HasValue) invQuery = invQuery.Where(i => i.TermId == termId.Value);

    var invoiceRows = await invQuery
        .Select(i => new
        {
            i.AmountDue,
            i.AmountPaid,
            StatusCode = i.InvoiceStatus!.Code,
        })
        .ToListAsync(ct);

    var summary = new FinanceSummaryDto
    {
        TermId = termId,
        TermName = termName,
        InvoiceCount = invoiceRows.Count,
        PaidInvoiceCount = invoiceRows.Count(r => r.StatusCode == "PAID"),
        PartiallyPaidInvoiceCount = invoiceRows.Count(r => r.StatusCode == "PARTIAL"),
        UnpaidInvoiceCount = invoiceRows.Count(r =>
            r.StatusCode == "ISSUED" || r.StatusCode == "OVERDUE"),
        TotalInvoiced = invoiceRows.Sum(r => r.AmountDue),
        TotalCollected = invoiceRows.Sum(r => r.AmountPaid),
    };
    summary.TotalOutstanding = summary.TotalInvoiced - summary.TotalCollected;

    var payQuery = db.Payments.AsQueryable();
    if (termId.HasValue)

```



```

    {
        payQuery = payQuery.Where(p =>
            p.Allocations.Any(a => a.Invoice!.TermId == termId.Value));
    }
    summary.PaymentCount = await payQuery.CountAsync(ct);
    summary.ByMethod = await payQuery
        .GroupBy(p => new { p.PaymentMethodId, p.PaymentMethod!.Name })
        .Select(g => new MethodBreakdownDto
        {
            PaymentMethodId = g.Key.PaymentMethodId,
            PaymentMethodName = g.Key.Name,
            TotalAmount = g.Sum(p => p.Amount),
            PaymentCount = g.Count(),
        })
        .OrderByDescending(b => b.TotalAmount)
        .ToListAsync(ct);

    var lineQuery = db.InvoiceLines.AsQueryable();
    if (termId.HasValue)
        lineQuery = lineQuery.Where(l => l.Invoice!.TermId == termId.Value);
    summary.ByCategory = await lineQuery
        .GroupBy(l => new { l.FeeCategoryId, l.FeeCategory!.Name })
        .Select(g => new CategoryBreakdownDto
        {
            FeeCategoryId = g.Key.FeeCategoryId,
            FeeCategoryName = g.Key.Name,
            TotalInvoiced = g.Sum(l => l.Amount - l.Discount),
        })
        .OrderByDescending(b => b.TotalInvoiced)
        .ToListAsync(ct);

    return summary;
}
}

```

6.4 DI registration

Excerpt — DependencyInjection.cs (sprint 6 additions)

```

services.AddScoped<IFeeScheduleService, FeeScheduleService>();
// InvoiceService is registered twice: as IInvoiceService for the UI,
// and as the concrete InvoiceService so PaymentService can call
// its internal RecomputeInvoiceTotalsAsync helper directly.
services.AddScoped<InvoiceService>();
services.AddScoped<IInvoiceService>(sp => sp.GetRequiredService<InvoiceService>());
services.AddScoped<IPaymentService, PaymentService>();

return services;

```

6.5 LookupService — four new methods

Excerpt — LookupService.cs (sprint 6 additions)

```

public async Task<IReadOnlyList<LookupDto>> GetFeeCategoriesAsync(Cancellation token ct

```

```
= default) =>
    await db.FeeCategories
        .OrderBy(c => c.DisplayOrder)
        .Select(c => new LookupDto { Id = c.Id, Name = c.Name, Code = c.Code })
        .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetPaymentMethodsAsync(CancellationTokentoken ct
= default) =>
    await db.PaymentMethods
        .OrderBy(m => m.DisplayOrder)
        .Select(m => new LookupDto { Id = m.Id, Name = m.Name, Code = m.Code })
        .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetInvoiceStatusesAsync(CancellationTokentoken
ct = default) =>
    await db.InvoiceStatuses
        .OrderBy(s => s.DisplayOrder)
        .Select(s => new LookupDto { Id = s.Id, Name = s.Name, Code = s.Code })
        .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetPaymentStatusesAsync(CancellationTokentoken
ct = default) =>
    await db.PaymentStatuses
        .OrderBy(s => s.DisplayOrder)
        .Select(s => new LookupDto { Id = s.Id, Name = s.Name, Code = s.Code })
        .ToListAsync(ct);
}
```

7. The EF Core migration

```
dotnet ef migrations add FeesInvoicesPayments \
  --project src/NaijaPrimeSchool.Infrastructure \
  --startup-project src/NaijaPrimeSchool.Web \
  --output-dir Persistence/Migrations
```

Excerpt — Up() of 20260512224258_FeesInvoicesPayments.cs

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    migrationBuilder.CreateTable(
        name: "FeeCategories",
        columns: table => new
        {
            Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
            Name = table.Column<string>(type: "nvarchar(60)", maxLength: 60,
nullable: false),
            Code = table.Column<string>(type: "nvarchar(10)", maxLength: 10,
nullable: false),
            DisplayOrder = table.Column<int>(type: "int", nullable: false),
            IsMandatoryByDefault = table.Column<bool>(type: "bit", nullable:
false),
            CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
            CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
            ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
            ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
            IsDeleted = table.Column<bool>(type: "bit", nullable: false),
            DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
            DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
        },
        constraints: table =>
        {
            table.PrimaryKey("PK_FeeCategories", x => x.Id);
        });

    migrationBuilder.CreateTable(
        name: "FeeSchedules",
        columns: table => new
        {
            Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
            TermId = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
            ClassLevelId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
            Title = table.Column<string>(type: "nvarchar(120)", maxLength: 120,
nullable: false),
            Notes = table.Column<string>(type: "nvarchar(500)", maxLength: 500,
nullable: true),
            IsPublished = table.Column<bool>(type: "bit", nullable: false),
```

```

        nullable: true),
        CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
        nullable: false),
        CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
        nullable: true),
        ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
        nullable: true),
        ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
        100, nullable: true),
        IsDeleted = table.Column<bool>(type: "bit", nullable: false),
        DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
        nullable: true),
        DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
        nullable: true)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_FeeSchedules", x => x.Id);
        table.ForeignKey(
            name: "FK_FeeSchedules_ClassLevels_ClassLevelId",
            column: x => x.ClassLevelId,
            principalTable: "ClassLevels",
            principalColumn: "Id",
            onDelete: ReferentialAction.Restrict);
        table.ForeignKey(
            name: "FK_FeeSchedules_Terms_TermId",
            column: x => x.TermId,
            principalTable: "Terms",
            principalColumn: "Id",
            onDelete: ReferentialAction.Restrict);
    });

    migrationBuilder.CreateTable(
        name: "InvoiceStatuses",
        columns: table => new
        {
            Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
            Name = table.Column<string>(type: "nvarchar(40)", maxLength: 40,
            nullable: false),
            Code = table.Column<string>(type: "nvarchar(20)", maxLength: 20,
            nullable: false),
            DisplayOrder = table.Column<int>(type: "int", nullable: false),
            CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
            nullable: false),
            CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
            nullable: true),
            ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
            nullable: true),
            ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
            100, nullable: true),
            IsDeleted = table.Column<bool>(type: "bit", nullable: false),
            DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
            nullable: true),
            DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,

```

```

nullable: true)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_InvoiceStatuses", x => x.Id);
    });

migrationBuilder.CreateTable(
    name: "PaymentMethods",
    columns: table => new
    {
        Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
        Name = table.Column<string>(type: "nvarchar(60)", maxLength: 60,
nullable: false),
        Code = table.Column<string>(type: "nvarchar(10)", maxLength: 10,
nullable: false),
        DisplayOrder = table.Column<int>(type: "int", nullable: false),
        RequiresReference = table.Column<bool>(type: "bit", nullable: false),
        CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
        CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
        ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
        ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
        IsDeleted = table.Column<bool>(type: "bit", nullable: false),
        DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
        DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_PaymentMethods", x => x.Id);
    });

migrationBuilder.CreateTable(
    name: "PaymentStatuses",
    columns: table => new
    {
        Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
        Name = table.Column<string>(type: "nvarchar(40)", maxLength: 40,
nullable: false),
        Code = table.Column<string>(type: "nvarchar(20)", maxLength: 20,
nullable: false),
        DisplayOrder = table.Column<int>(type: "int", nullable: false),
        CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
        CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
        ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
        ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),

```

```

        IsDeleted = table.Column<bool>(type: "bit", nullable: false),
        DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
        DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_PaymentStatuses", x => x.Id);
    });

migrationBuilder.CreateTable(
    name: "FeeScheduleItems",
    columns: table => new
    {
        Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
        FeeScheduleId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
        FeeCategoryId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
        Description = table.Column<string>(type: "nvarchar(160)", maxLength:
160, nullable: false),
        Amount = table.Column<decimal>(type: "decimal(12,2)", precision: 12,
scale: 2, nullable: false),
        IsMandatory = table.Column<bool>(type: "bit", nullable: false),
        DisplayOrder = table.Column<int>(type: "int", nullable: false),
        CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
        CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
        ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
        ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
        IsDeleted = table.Column<bool>(type: "bit", nullable: false),
        DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
        DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_FeeScheduleItems", x => x.Id);
        table.ForeignKey(
            name: "FK_FeeScheduleItems_FeeCategories_FeeCategoryId",
            column: x => x.FeeCategoryId,
            principalTable: "FeeCategories",
            principalColumn: "Id",
            onDelete: ReferentialAction.Restrict);
        table.ForeignKey(
            name: "FK_FeeScheduleItems_FeeSchedules_FeeScheduleId",
            column: x => x.FeeScheduleId,
            principalTable: "FeeSchedules",
            principalColumn: "Id",
            onDelete: ReferentialAction.Cascade);
    });

```

```

    });

    migrationBuilder.CreateTable(
        name: "Invoices",
        columns: table => new
        {
            Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
            StudentId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
            TermId = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
            SchoolClassId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
            InvoiceStatusId = table.Column<Guid>(type: "uniqueidentifier",
nullable: false),
            InvoiceNumber = table.Column<string>(type: "nvarchar(40)", maxLength:
40, nullable: false),
            IssuedOn = table.Column<DateOnly>(type: "date", nullable: false),
            DueDate = table.Column<DateOnly>(type: "date", nullable: true),
            Subtotal = table.Column<decimal>(type: "decimal(12,2)", precision: 12,
scale: 2, nullable: false),
            DiscountTotal = table.Column<decimal>(type: "decimal(12,2)", precision:
12, scale: 2, nullable: false),
            AmountDue = table.Column<decimal>(type: "decimal(12,2)", precision: 12,
scale: 2, nullable: false),
            AmountPaid = table.Column<decimal>(type: "decimal(12,2)", precision:
12, scale: 2, nullable: false),
            Notes = table.Column<string>(type: "nvarchar(500)", maxLength: 500,
nullable: true),
            CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
            CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
            ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
            ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
            IsDeleted = table.Column<bool>(type: "bit", nullable: false),
            DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
            DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
        },
        constraints: table =>
        {
            table.PrimaryKey("PK_Invoices", x => x.Id);
            table.ForeignKey(
                name: "FK_Invoices_InvoiceStatuses_InvoiceStatusId",
                column: x => x.InvoiceStatusId,
                principalTable: "InvoiceStatuses",
                principalColumn: "Id",
                onDelete: ReferentialAction.Restrict);
            table.ForeignKey(
                name: "FK_Invoices_SchoolClasses_SchoolClassId",
                column: x => x.SchoolClassId,
                principalTable: "SchoolClasses",

```

```

        principalColumn: "Id",
        onDelete: ReferentialAction.Restrict);
table.ForeignKey(
    name: "FK_Invoices_Students_StudentId",
    column: x => x.StudentId,
    principalTable: "Students",
    principalColumn: "Id",
    onDelete: ReferentialAction.Restrict);
table.ForeignKey(
    name: "FK_Invoices_Terms_TermId",
    column: x => x.TermId,
    principalTable: "Terms",
    principalColumn: "Id",
    onDelete: ReferentialAction.Restrict);
});

migrationBuilder.CreateTable(
    name: "Payments",
    columns: table => new
    {
        Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
        StudentId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
        PaymentMethodId = table.Column<Guid>(type: "uniqueidentifier",
nullable: false),
        PaymentStatusId = table.Column<Guid>(type: "uniqueidentifier",
nullable: false),
        ReceiptNumber = table.Column<string>(type: "nvarchar(40)", maxLength:
40, nullable: false),
        PaidOn = table.Column<DateOnly>(type: "date", nullable: false),
        Amount = table.Column<decimal>(type: "decimal(12,2)", precision: 12,
scale: 2, nullable: false),
        Reference = table.Column<string>(type: "nvarchar(120)", maxLength: 120,
nullable: true),
        Notes = table.Column<string>(type: "nvarchar(300)", maxLength: 300,
nullable: true),
        CollectedById = table.Column<Guid>(type: "uniqueidentifier", nullable:
true),
        CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
        CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
        ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
        ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
        IsDeleted = table.Column<bool>(type: "bit", nullable: false),
        DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
        DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_Payments", x => x.Id);
    }
);

```



```

        table.ForeignKey(
            name: "FK_Payments_PaymentMethods_PaymentMethodId",
            column: x => x.PaymentMethodId,
            principalTable: "PaymentMethods",
            principalColumn: "Id",
            onDelete: ReferentialAction.Restrict);
        table.ForeignKey(
            name: "FK_Payments_PaymentStatuses_PaymentStatusId",
            column: x => x.PaymentStatusId,
            principalTable: "PaymentStatuses",
            principalColumn: "Id",
            onDelete: ReferentialAction.Restrict);
        table.ForeignKey(
            name: "FK_Payments_Students_StudentId",
            column: x => x.StudentId,
            principalTable: "Students",
            principalColumn: "Id",
            onDelete: ReferentialAction.Restrict);
        table.ForeignKey(
            name: "FK_Payments_Users_CollectedById",
            column: x => x.CollectedById,
            principalTable: "Users",
            principalColumn: "Id",
            onDelete: ReferentialAction.SetNull);
    });

    migrationBuilder.CreateTable(
        name: "InvoiceLines",
        columns: table => new
        {
            Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
            InvoiceId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
            FeeCategoryId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
            FeeScheduleItemId = table.Column<Guid>(type: "uniqueidentifier",
nullable: true),
            Description = table.Column<string>(type: "nvarchar(160)", maxLength:
160, nullable: false),
            Amount = table.Column<decimal>(type: "decimal(12,2)", precision: 12,
scale: 2, nullable: false),
            Discount = table.Column<decimal>(type: "decimal(12,2)", precision: 12,
scale: 2, nullable: false),
            CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
            CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
            ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
            ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
            IsDeleted = table.Column<bool>(type: "bit", nullable: false),
            DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
            DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,

```

```

nullable: true)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_InvoiceLines", x => x.Id);
        table.ForeignKey(
            name: "FK_InvoiceLines_FeeCategories_FeeCategoryId",
            column: x => x.FeeCategoryId,
            principalTable: "FeeCategories",
            principalColumn: "Id",
            onDelete: ReferentialAction.Restrict);
        table.ForeignKey(
            name: "FK_InvoiceLines_FeeScheduleItems_FeeScheduleItemId",
            column: x => x.FeeScheduleItemId,
            principalTable: "FeeScheduleItems",
            principalColumn: "Id",
            onDelete: ReferentialAction.SetNull);
        table.ForeignKey(
            name: "FK_InvoiceLines_Invoices_InvoiceId",
            column: x => x.InvoiceId,
            principalTable: "Invoices",
            principalColumn: "Id",
            onDelete: ReferentialAction.Cascade);
    });

migrationBuilder.CreateTable(
    name: "PaymentAllocations",
    columns: table => new
    {
        Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
        PaymentId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
        InvoiceId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
        AmountApplied = table.Column<decimal>(type: "decimal(12,2)", precision:
12, scale: 2, nullable: false),
        CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
        CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
        ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
        ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
        IsDeleted = table.Column<bool>(type: "bit", nullable: false),
        DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
        DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_PaymentAllocations", x => x.Id);
        table.ForeignKey(
            name: "FK_PaymentAllocations_Invoices_InvoiceId",

```

```

        column: x => x.InvoiceId,
        principalTable: "Invoices",
        principalColumn: "Id",
        onDelete: ReferentialAction.Restrict);
    table.ForeignKey(
        name: "FK_PaymentAllocations_Payments_PaymentId",
        column: x => x.PaymentId,
        principalTable: "Payments",
        principalColumn: "Id",
        onDelete: ReferentialAction.Cascade);
    });

migrationBuilder.CreateIndex(
    name: "IX_FeeCategories_Code",
    table: "FeeCategories",
    column: "Code",
    unique: true);

migrationBuilder.CreateIndex(
    name: "IX_FeeCategories_Name",
    table: "FeeCategories",
    column: "Name",
    unique: true);

migrationBuilder.CreateIndex(
    name: "IX_FeeScheduleItems_FeeCategoryId",
    table: "FeeScheduleItems",
    column: "FeeCategoryId");

migrationBuilder.CreateIndex(
    name: "IX_FeeScheduleItems_FeeScheduleId_DisplayOrder",
    table: "FeeScheduleItems",
    columns: new[] { "FeeScheduleId", "DisplayOrder" });

migrationBuilder.CreateIndex(
    name: "IX_FeeScheduleItems_IsDeleted",
    table: "FeeScheduleItems",
    column: "IsDeleted");

migrationBuilder.CreateIndex(
    name: "IX_FeeSchedules_ClassLevelId",
    table: "FeeSchedules",
    column: "ClassLevelId");

migrationBuilder.CreateIndex(
    name: "IX_FeeSchedules_IsDeleted",
    table: "FeeSchedules",
    column: "IsDeleted");

migrationBuilder.CreateIndex(
    name: "IX_FeeSchedules_TermId_ClassLevelId",
    table: "FeeSchedules",
    columns: new[] { "TermId", "ClassLevelId" },
    unique: true);

```

```
migrationBuilder.CreateIndex(  
    name: "IX_InvoiceLines_FeeCategoryId",  
    table: "InvoiceLines",  
    column: "FeeCategoryId");  
  
migrationBuilder.CreateIndex(  
    name: "IX_InvoiceLines_FeeScheduleItemId",  
    table: "InvoiceLines",  
    column: "FeeScheduleItemId");  
  
migrationBuilder.CreateIndex(  
    name: "IX_InvoiceLines_InvoiceId",  
    table: "InvoiceLines",  
    column: "InvoiceId");  
  
migrationBuilder.CreateIndex(  
    name: "IX_InvoiceLines_IsDeleted",  
    table: "InvoiceLines",  
    column: "IsDeleted");  
  
migrationBuilder.CreateIndex(  
    name: "IX_Invoices_InvoiceNumber",  
    table: "Invoices",  
    column: "InvoiceNumber",  
    unique: true);  
  
migrationBuilder.CreateIndex(  
    name: "IX_Invoices_InvoiceStatusId",  
    table: "Invoices",  
    column: "InvoiceStatusId");  
  
migrationBuilder.CreateIndex(  
    name: "IX_Invoices_IsDeleted",  
    table: "Invoices",  
    column: "IsDeleted");  
  
migrationBuilder.CreateIndex(  
    name: "IX_Invoices_SchoolClassId",  
    table: "Invoices",  
    column: "SchoolClassId");  
  
migrationBuilder.CreateIndex(  
    name: "IX_Invoices_StudentId_TermId",  
    table: "Invoices",  
    columns: new[] { "StudentId", "TermId" });  
  
migrationBuilder.CreateIndex(  
    name: "IX_Invoices_TermId",  
    table: "Invoices",  
    column: "TermId");  
  
migrationBuilder.CreateIndex(  
    name: "IX_InvoiceStatuses_Code",
```

```
        table: "InvoiceStatuses",
        column: "Code",
        unique: true);

migrationBuilder.CreateIndex(
    name: "IX_InvoiceStatuses_Name",
    table: "InvoiceStatuses",
    column: "Name",
    unique: true);

migrationBuilder.CreateIndex(
    name: "IX_PaymentAllocations_InvoiceId",
    table: "PaymentAllocations",
    column: "InvoiceId");

migrationBuilder.CreateIndex(
    name: "IX_PaymentAllocations_IsDeleted",
    table: "PaymentAllocations",
    column: "IsDeleted");

migrationBuilder.CreateIndex(
    name: "IX_PaymentAllocations_PaymentId_InvoiceId",
    table: "PaymentAllocations",
    columns: new[] { "PaymentId", "InvoiceId" },
    unique: true);

migrationBuilder.CreateIndex(
    name: "IX_PaymentMethods_Code",
    table: "PaymentMethods",
    column: "Code",
    unique: true);

migrationBuilder.CreateIndex(
    name: "IX_PaymentMethods_Name",
    table: "PaymentMethods",
    column: "Name",
    unique: true);

migrationBuilder.CreateIndex(
    name: "IX_Payments_CollectedById",
    table: "Payments",
    column: "CollectedById");

migrationBuilder.CreateIndex(
    name: "IX_Payments_IsDeleted",
    table: "Payments",
    column: "IsDeleted");

migrationBuilder.CreateIndex(
    name: "IX_Payments_PaidOn",
    table: "Payments",
    column: "PaidOn");

migrationBuilder.CreateIndex(
```

```

        name: "IX_Payments_PaymentMethodId",
        table: "Payments",
        column: "PaymentMethodId");

migrationBuilder.CreateIndex(
    name: "IX_Payments_PaymentStatusId",
    table: "Payments",
    column: "PaymentStatusId");

migrationBuilder.CreateIndex(
    name: "IX_Payments_ReceiptNumber",
    table: "Payments",
    column: "ReceiptNumber",
    unique: true);

migrationBuilder.CreateIndex(
    name: "IX_Payments_StudentId",
    table: "Payments",
    column: "StudentId");

migrationBuilder.CreateIndex(
    name: "IX_PaymentStatuses_Code",
    table: "PaymentStatuses",
    column: "Code",
    unique: true);

migrationBuilder.CreateIndex(
    name: "IX_PaymentStatuses_Name",
    table: "PaymentStatuses",
    column: "Name",
    unique: true);
}

/// <inheritdoc />
protected override void Down(MigrationBuilder migrationBuilder)

```

8. Seeding the finance lookups

Excerpt — *SeedFinanceLookupsAsync*

```
private static async Task SeedFinanceLookupsAsync(ApplicationDbContext db,
CancellationTokens ct)
{
    if (!await db.FeeCategories.IgnoreQueryFilters().AnyAsync(ct))
    {
        (string Name, string Code, bool Mandatory)[] categories =
        [
            ("Tuition", "TUI", true),
            ("Development Levy", "DEV", true),
            ("Examination", "EXAM", true),
            ("Books", "BOOK", false),
            ("Uniforms", "UNIF", false),
            ("Transport", "TRAN", false),
            ("Meals", "MEAL", false),
            ("Boarding", "BRD", false),
            ("PTA Levy", "PTA", true),
            ("Other", "OTH", false),
        ];
        for (var i = 0; i < categories.Length; i++)
        {
            var c = categories[i];
            db.FeeCategories.Add(new FeeCategory
            {
                Name = c.Name,
                Code = c.Code,
                DisplayOrder = i + 1,
                IsMandatoryByDefault = c.Mandatory,
            });
        }
    }

    if (!await db.PaymentMethods.IgnoreQueryFilters().AnyAsync(ct))
    {
        (string Name, string Code, bool RequiresRef)[] methods =
        [
            ("Cash", "CASH", false),
            ("Bank Transfer", "BANK", true),
            ("POS", "POS", true),
            ("Cheque", "CHQ", true),
            ("Mobile Money", "MMNY", true),
            ("Online Payment", "ONL", true),
        ];
        for (var i = 0; i < methods.Length; i++)
        {
            var m = methods[i];
            db.PaymentMethods.Add(new PaymentMethod
            {
                Name = m.Name,
                Code = m.Code,
                DisplayOrder = i + 1,
                RequiresReference = m.RequiresRef,
            });
        }
    }
}
```

```

        });
    }
}

if (!await db.InvoiceStatuses.IgnoreQueryFilters().AnyAsync(ct))
{
    (string Name, string Code)[] statuses =
    [
        ("Draft", "DRAFT"),
        ("Issued", "ISSUED"),
        ("Partially Paid", "PARTIAL"),
        ("Paid", "PAID"),
        ("Overdue", "OVERDUE"),
        ("Cancelled", "CANCELLED"),
    ];
    for (var i = 0; i < statuses.Length; i++)
    {
        db.InvoiceStatuses.Add(new InvoiceStatus
        {
            Name = statuses[i].Name,
            Code = statuses[i].Code,
            DisplayOrder = i + 1,
        });
    }
}

if (!await db.PaymentStatuses.IgnoreQueryFilters().AnyAsync(ct))
{
    (string Name, string Code)[] statuses =
    [
        ("Pending", "PENDING"),
        ("Confirmed", "CONFIRMED"),
        ("Bounced", "BOUNCED"),
        ("Refunded", "REFUNDED"),
    ];
    for (var i = 0; i < statuses.Length; i++)
    {
        db.PaymentStatuses.Add(new PaymentStatus
        {
            Name = statuses[i].Name,
            Code = statuses[i].Code,
            DisplayOrder = i + 1,
        });
    }
}

await db.SaveChangesAsync(ct);
}

private static async Task SeedResultsLookupsAsync(ApplicationDbContext db,
Cancellation token ct)

```

What gets seeded:

- FeeCategories — Tuition (TUI), Development Levy (DEV), Examination (EXAM), Books (BOOK), Uniforms (UNIF), Transport (TRAN), Meals (MEAL), Boarding (BRD), PTA Levy (PTA), Other (OTH). Each has an IsMandatoryByDefault flag.
- PaymentMethods — Cash (CASH), Bank Transfer (BANK), POS (POS), Cheque (CHQ), Mobile Money (MMNY), Online Payment (ONL). Each has a RequiresReference flag that the UI can use to demand a reference number.
- InvoiceStatuses — Draft (DRAFT), Issued (ISSUED), Partially Paid (PARTIAL), Paid (PAID), Overdue (OVERDUE), Cancelled (CANCELLED). The service layer keys off Code, not Id, when transitioning.
- PaymentStatuses — Pending (PENDING), Confirmed (CONFIRMED), Bounced (BOUNCED), Refunded (REFUNDED).

9. The Razor pages

```
src/NaijaPrimeSchool.Web/Components/Pages/Finance/
├── FeeSchedules.razor          <- /fees
├── FeeScheduleDetail.razor     <- /fees/{id}
├── Invoices.razor             <- /invoices
├── IssueInvoices.razor        <- /invoices/issue
├── InvoiceDetail.razor         <- /invoices/{id}
├── Payments.razor             <- /payments
├── RecordPayment.razor        <- /payments/new
├── PaymentDetail.razor        <- /payments/{id}
└── FinanceDashboard.razor     <- /finance
```

Every page is gated to SuperAdmin + HeadTeacher + SchoolBursar. Sprint 6 is the first sprint where the SchoolBursar role moves from placeholder navigation to a fully realised workspace.

9.1 FeeSchedules.razor — the schedule list

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Finance/FeeSchedules.razor

```
@page "/fees"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},{Roles.SchoolBursar}")]
@rendermode InteractiveServer

@inject IFeeScheduleService FeeScheduleService
@inject ITermService TermService
@inject ILookupService LookupService
@inject NavigationManager Nav
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Fee schedules · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Fee schedules</h1>
        <p>Define the per-(term, class level) fee structure that drives every invoice issued to a pupil.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="New schedule" Icon="add" ButtonStyle="ButtonStyle.Primary"
            Click="@(() => OpenForm(null))" />
    </div>
</div>

<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <div class="nps-filter-bar">
        <div class="nps-field">
            <label>Term</label>
            <RadzenDropDown TValue="Guid?" Data="@terms" @bind-Value="termId"
                ValueProperty="Id" TextProperty="TermTypeName"
                AllowClear="true" Placeholder="All terms"
                Change="@(_ => LoadAsync())" Style="min-width: 220px;">
```

```

        <Template Context="t">
            @{ var dto = (TermDto)t; }
            @dto.TermTypeName - @dto.SessionName
        </Template>
    </RadzenDropDown>
</div>
<div class="nps-field">
    <label>Class level</label>
    <RadzenDropDown TValue="Guid?" Data="@classLevels" TextProperty="Name"
ValueProperty="Id"
                        @bind-Value="classLevelId" AllowClear="true" Placeholder="All
levels"
                        Change="@(_ => LoadAsync())" Style="min-width: 200px;" />
</div>
<div class="nps-field">
    <label>Status</label>
    <RadzenDropDown TValue="bool?" Data="@statusOptions" TextProperty="Text"
ValueProperty="Value"
                        @bind-Value="isPublished" AllowClear="true" Placeholder="All"
                        Change="@(_ => LoadAsync())" Style="min-width: 160px;" />
</div>
</div>
</RadzenCard>

<RadzenCard class="nps-card">
    <RadzenDataGrid TItem="FeeScheduleDto" Data="@schedules" IsLoading="@loading"
AllowPaging="true" PageSize="25" EmptyText="No fee schedules match your
filters.">
        <Columns>
            <RadzenDataGridColumn TItem="FeeScheduleDto" Title="Title">
                <Template Context="s">
                    <strong>@s.Title</strong>
                    <br />
                    <small style="color: var(--nps-ink-500);">@s.ClassLevelName ·
@s.TermName</small>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="FeeScheduleDto" Title="Items" Width="100px"
Property="ItemCount" />
            <RadzenDataGridColumn TItem="FeeScheduleDto" Title="Total" Width="140px">
                <Template Context="s">
                    <strong>@s.TotalAmount.ToString("N2")</strong>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="FeeScheduleDto" Title="Invoices issued"
Width="140px" Property="InvoicesIssued" />
            <RadzenDataGridColumn TItem="FeeScheduleDto" Title="Status" Width="120px">
                <Template Context="s">
                    @if (s.IsPublished)
                    {
                        <RadzenBadge Text="Published" BadgeStyle="BadgeStyle.Success"
Variant="Variant.Flat" />
                    }
                    else
                    {

```

```

                <RadzenBadge Text="Draft" BadgeStyle="BadgeStyle.Secondary"
Variant="Variant.Flat" />
            }
        </Template>
    </RadzenDataGridColumn>
    <RadzenDataGridColumn TItem="FeeScheduleDto" Title="Actions" Sortable="false"
        Width="200px" TextAlign="TextAlign.Right">
        <Template Context="s">
            <div class="nps-inline-actions">
                <RadzenButton Icon="edit_note" Size="ButtonSize.Small"
Variant="Variant.Flat"
                    ButtonStyle="ButtonStyle.Primary" title="Edit items"
                    Click="@(() => Nav.NavigateTo($"/fees/{s.Id}")" />
                @if (s.IsPublished)
                {
                    <RadzenButton Icon="lock_open" Size="ButtonSize.Small"
Variant="Variant.Flat"
                        ButtonStyle="ButtonStyle.Warning"
                        title="Unpublish"
                        Click="@(() => UnpublishAsync(s))" />
                }
                else
                {
                    <RadzenButton Icon="lock" Size="ButtonSize.Small"
Variant="Variant.Flat"
                        ButtonStyle="ButtonStyle.Success" title="Publish"
                        Click="@(() => PublishAsync(s))" />
                }
                <RadzenButton Icon="delete" Size="ButtonSize.Small"
Variant="Variant.Flat"
                    ButtonStyle="ButtonStyle.Danger" title="Delete"
                    Click="@(() => DeleteAsync(s))" />
            </div>
        </Template>
    </RadzenDataGridColumn>
</Columns>
</RadzenDataGrid>
</RadzenCard>

@if (showForm)
{
    <RadzenCard class="nps-card" Style="margin-top:1rem;">
        <h3 style="margin: 0 0 1rem;">New fee schedule</h3>
        <RadzenTemplateForm TItem="ScheduleFormModel" Data="@form" Submit="@SaveAsync">
            <DataAnnotationsValidator />
            <div class="nps-form-grid">
                <div class="nps-field">
                    <label>Term *</label>
                    <RadzenDropDown TValue="Guid?" Data="@terms" @bind-Value="form.TermId"
                        ValueProperty="Id" TextProperty="TermTypeName"
                        Placeholder="Pick a term">
                        <Template Context="t">
                            @{ var dto = (TermDto)t; }
                            @dto.TermTypeName - @dto.SessionName
                        </Template>
                    </RadzenDropDown>
                </div>
            </div>
        </RadzenTemplateForm>
    </RadzenCard>
}

```

```

        </RadzenDropDown>
    </div>
    <div class="nps-field">
        <label>Class level *</label>
        <RadzenDropDown TValue="Guid?" Data="@classLevels" TextProperty="Name"
ValueProperty="Id"
                        @bind-Value="form.ClassLevelId" Placeholder="Pick a
level" />
    </div>
    <div class="nps-field nps-span-2">
        <label>Title *</label>
        <RadzenTextBox @bind-Value="form.Title"
                        Placeholder="e.g. Primary 1 – First Term 2025/2026 fees"
/>
        <ValidationMessage For="() => form.Title" />
    </div>
    <div class="nps-field nps-span-2">
        <label>Notes</label>
        <RadzenTextArea @bind-Value="form.Notes" Rows="2" />
    </div>
</div>
<div class="nps-form-actions">
    <RadzenButton Text="Cancel" ButtonType="ButtonType.Button"
                  ButtonStyle="ButtonStyle.Light" Variant="Variant.Flat"
                  Click="@(() => showForm = false)" />
    <RadzenButton Text="@((saving ? "Saving..." : "Create schedule"))"
                  Icon="save" ButtonType="ButtonType.Submit"
                  ButtonStyle="ButtonStyle.Primary" Disabled="@saving" />
</div>
</RadzenTemplateForm>
</RadzenCard>
}

@code {
    private IReadOnlyList<FeeScheduleDto> schedules = [];
    private List<TermDto> terms = [];
    private IReadOnlyList<LookupDto> classLevels = [];

    private bool loading;
    private bool saving;
    private bool showForm;

    private Guid? termId;
    private Guid? classLevelId;
    private bool? isPublished;

    private record StatusOption(string Text, bool? Value);
    private readonly List<StatusOption> statusOptions =
    [
        new StatusOption("Draft", false),
        new StatusOption("Published", true),
    ];

    private ScheduleFormModel form = new();

```

```

private class ScheduleFormModel
{
    public Guid? TermId { get; set; }
    public Guid? ClassLevelId { get; set; }

    [System.ComponentModel.DataAnnotations.Required,
System.ComponentModel.DataAnnotations.StringLength(120)]
    public string Title { get; set; } = string.Empty;

    [System.ComponentModel.DataAnnotations.StringLength(500)]
    public string? Notes { get; set; }
}

protected override async Task OnInitializedAsync()
{
    terms = (await TermService.ListAsync()).ToList();
    termId = terms.FirstOrDefault(t => t.IsCurrent)?.Id;
    classLevels = await LookupService.GetClassLevelsAsync();
    await LoadAsync();
}

private async Task LoadAsync()
{
    loading = true;
    try
    {
        schedules = await FeeScheduleService.ListAsync(new FeeScheduleFilter
        {
            TermId = termId,
            ClassLevelId = classLevelId,
            IsPublished = isPublished,
        });
    }
    finally
    {
        loading = false;
        StateHasChanged();
    }
}

private void OpenForm(FeeScheduleDto? _)
{
    form = new ScheduleFormModel
    {
        TermId = termId ?? terms.FirstOrDefault(t => t.IsCurrent)?.Id,
        ClassLevelId = classLevelId,
    };
    showForm = true;
}

private async Task SaveAsync()
{
    if (form.TermId is null || form.ClassLevelId is null)

```

```

    {
        Notification.Notify(NotificationSeverity.Warning, "Missing fields",
            "Pick a term and a class level.");
        return;
    }

    saving = true;
    try
    {
        var result = await FeeScheduleService.CreateAsync(new CreateFeeScheduleRequest
        {
            TermId = form.TermId.Value,
            ClassLevelId = form.ClassLevelId.Value,
            Title = form.Title,
            Notes = form.Notes,
        });

        if (result.Succeeded && result.Data != Guid.Empty)
        {
            Notification.Notify(NotificationSeverity.Success, "Created", "Fee schedule
created.");
            showForm = false;
            Nav.NavigateTo($" /fees/{result.Data}");
        }
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
        }
    }
    finally
    {
        saving = false;
    }
}

private async Task PublishAsync(FeeScheduleDto s)
{
    var result = await FeeScheduleService.PublishAsync(s.Id);
    Show(result, $"{s.Title}' published.");
    await LoadAsync();
}

private async Task UnpublishAsync(FeeScheduleDto s)
{
    var result = await FeeScheduleService.UnpublishAsync(s.Id);
    Show(result, $"{s.Title}' unpublished.");
    await LoadAsync();
}

private async Task DeleteAsync(FeeScheduleDto s)
{
    var confirm = await Dialog.Confirm($"Delete fee schedule '{s.Title}'?",
        "Delete fee schedule",

```

```

        new ConfirmOptions { OkButtonText = "Delete", CancelButtonText = "Cancel" });
        if (confirm != true) return;

        var result = await FeeScheduleService.SoftDeleteAsync(s.Id);
        Show(result, $"{s.Title}' deleted.");
        await LoadAsync();
    }

    private void Show(ActionResult result, string success)
    {
        if (result.Succeeded)
            Notification.Notify(NotificationSeverity.Success, "Done", success);
        else
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
    }
}

```

9.2 FeeScheduleDetail.razor — schedule + items

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Finance/FeeScheduleDetail.razor

```

@page "/fees/{Id:guid}"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},
{Roles.SchoolBursar}")]]
@rendermode InteractiveServer

@inject IFeeScheduleService FeeScheduleService
@inject ILookupService LookupService
@inject NavigationManager Nav
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Fee schedule · Naija Prime School</PageTitle>

@if (loading)
{
    <p>Loading fee schedule...</p>
}
else if (detail is null)
{
    <RadzenAlert Severity="AlertSeverity.Warning">Fee schedule not found.</RadzenAlert>
}
else
{
    <div class="nps-page-header">
        <div>
            <h1>@detail.Schedule.Title</h1>
            <p>@detail.Schedule.ClassLevelName · @detail.Schedule.TermName</p>
        </div>
        <div class="nps-page-actions">
            @if (detail.Schedule.IsPublished)
            {
                <RadzenBadge Text="Published" BadgeStyle="BadgeStyle.Success"

```



```

Variant="Variant.Flat" />
        <RadzenButton Text="Unpublish" Icon="lock_open" Variant="Variant.Flat"
            ButtonStyle="ButtonStyle.Warning" Click="@UnpublishAsync" />
    }
    else
    {
        <RadzenButton Text="Publish" Icon="lock" Variant="Variant.Flat"
            ButtonStyle="ButtonStyle.Success" Click="@PublishAsync" />
    }
    <RadzenButton Text="Back" Icon="arrow_back" Variant="Variant.Flat"
        ButtonStyle="ButtonStyle.Light" Click="@(() =>
Nav.NavigateTo("/fees"))" />
    </div>
</div>

<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <div class="nps-form-grid">
        <div class="nps-field nps-span-2">
            <label>Title</label>
            <RadzenTextBox @bind-Value="title"
Disabled="@detail.Schedule.IsPublished" />
        </div>
        <div class="nps-field nps-span-2">
            <label>Notes</label>
            <RadzenTextArea @bind-Value="notes" Rows="2"
Disabled="@detail.Schedule.IsPublished" />
        </div>
    </div>
    <div class="nps-form-actions">
        <RadzenButton Text="@($savingHeader ? "Saving..." : "Save header")" Icon="save"
            ButtonStyle="ButtonStyle.Primary"
            Disabled="@($savingHeader || detail.Schedule.IsPublished)"
            Click="@SaveHeaderAsync" />
    </div>
</RadzenCard>

<RadzenCard class="nps-card">
    <div style="display:flex; justify-content: space-between; align-items: center;
margin-bottom: 1rem;">
        <h3 style="margin:0;">Line items (total <strong>@detail.Items.Sum(i =>
i.Amount).ToString("N2")</strong></h3>
        <RadzenButton Text="Add item" Icon="add" ButtonStyle="ButtonStyle.Primary"
            Disabled="@detail.Schedule.IsPublished"
            Click="@OpenAddItem" />
    </div>

    @if (detail.Items.Count == 0)
    {
        <p style="color: var(--nps-ink-500);">No items yet. Add at least one before
publishing.</p>
    }
    else
    {
        <RadzenDataGrid TItem="FeeScheduleItemDto" Data="@detail.Items"
AllowPaging="false">

```

```

        <Columns>
            <RadzenDataGridColumn TItem="FeeScheduleItemDto" Title="#" Width="60px"
Property="DisplayOrder" />
            <RadzenDataGridColumn TItem="FeeScheduleItemDto" Title="Category">
                <Template Context="i">
                    <strong>@i.FeeCategoryName</strong>
                    <br />
                    <small style="color:
var(--nps-ink-500);">@i.FeeCategoryCode</small>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="FeeScheduleItemDto" Title="Description"
Property="Description" />
            <RadzenDataGridColumn TItem="FeeScheduleItemDto" Title="Amount"
Width="140px">
                <Template Context="i">
                    <strong>@i.Amount.ToString("N2")</strong>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="FeeScheduleItemDto" Title="Mandatory"
Width="120px">
                <Template Context="i">
                    @if (i.IsMandatory)
                    {
                        <RadzenBadge Text="Yes" BadgeStyle="BadgeStyle.Success"
Variant="Variant.Flat" />
                    }
                    else
                    {
                        <RadzenBadge Text="Optional"
BadgeStyle="BadgeStyle.Secondary" Variant="Variant.Flat" />
                    }
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="FeeScheduleItemDto" Title="Actions"
Width="160px"
                Sortable="false" TextAlign="TextAlign.Right">
                <Template Context="i">
                    <div class="nps-inline-actions">
                        <RadzenButton Icon="edit" Size="ButtonSize.Small"
Variant="Variant.Flat"
                            ButtonStyle="ButtonStyle.Light"
                            Disabled="@detail.Schedule.IsPublished"
                            Click="@(() => OpenEditItem(i))" title="Edit"
                        />
                        <RadzenButton Icon="delete" Size="ButtonSize.Small"
Variant="Variant.Flat"
                            ButtonStyle="ButtonStyle.Danger"
                            Disabled="@detail.Schedule.IsPublished"
                            Click="@(() => RemoveItemAsync(i))"
                            title="Remove" />
                    </div>
                </Template>
            </RadzenDataGridColumn>
        </Columns>

```

```

        </RadzenDataGrid>
    }

    @if (showItemForm)
    {
        <RadzenCard class="nps-card" Style="margin-top:1rem; background: var(--nps-
green-200);">
            <h3 style="margin: 0 0 1rem;">@(itemEditingId.HasValue ? "Edit item" : "Add
item")</h3>
            <div class="nps-form-grid">
                <div class="nps-field">
                    <label>Category *</label>
                    <RadzenDropDown TValue="Guid?" Data="@feeCategories"
TextProperty="Name" ValueProperty="Id"
                    @bind-Value="itemForm.FeeCategoryId"
Placeholder="Pick a category" />
                </div>
                <div class="nps-field">
                    <label>Amount *</label>
                    <RadzenNumeric TValue="decimal" @bind-Value="itemForm.Amount"
Min="0" Format="N2" />
                </div>
                <div class="nps-field nps-span-2">
                    <label>Description *</label>
                    <RadzenTextBox @bind-Value="itemForm.Description" />
                </div>
                <div class="nps-field">
                    <label>Mandatory</label>
                    <RadzenSwitch @bind-Value="itemForm.IsMandatory" />
                </div>
                <div class="nps-field">
                    <label>Display order</label>
                    <RadzenNumeric TValue="int" @bind-Value="itemForm.DisplayOrder"
Min="0" />
                </div>
            </div>
            <div class="nps-form-actions">
                <RadzenButton Text="Cancel" ButtonType="ButtonType.Button"
                ButtonStyle="ButtonStyle.Light" Variant="Variant.Flat"
                Click="@(() => showItemForm = false)" />
                <RadzenButton Text="@(itemSaving ? "Saving..." : "Save")"
                Icon="save" ButtonStyle="ButtonStyle.Primary"
                Disabled="@itemSaving" Click="@SaveItemAsync" />
            </div>
        </RadzenCard>
    }
</RadzenCard>
}

@code {
    [Parameter] public Guid Id { get; set; }

    private FeeScheduleDetailDto? detail;
    private bool loading = true;

```

```

private string title = string.Empty;
private string? notes;
private bool savingHeader;

private IReadOnlyList<LookupDto> feeCategories = [];

private bool showItemForm;
private bool itemSaving;
private Guid? itemEditingId;
private ItemFormModel itemForm = new();

private class ItemFormModel
{
    public Guid? FeeCategoryId { get; set; }
    public string Description { get; set; } = string.Empty;
    public decimal Amount { get; set; }
    public bool IsMandatory { get; set; } = true;
    public int DisplayOrder { get; set; }
}

protected override async Task OnInitializedAsync()
{
    feeCategories = await LookupService.GetFeeCategoriesAsync();
    await LoadAsync();
}

private async Task LoadAsync()
{
    loading = true;
    try
    {
        {
            detail = await FeeScheduleService.GetByIdAsync(Id);
            if (detail is not null)
            {
                title = detail.Schedule.Title;
                notes = detail.Schedule.Notes;
            }
        }
    }
    finally
    {
        loading = false;
    }
}

private async Task SaveHeaderAsync()
{
    savingHeader = true;
    try
    {
        {
            var result = await FeeScheduleService.UpdateAsync(new UpdateFeeScheduleRequest
            {
                Id = Id,
                Title = title,
                Notes = notes,
            });
        }
    }
}

```

```

        });
        Show(result, "Header saved.");
        if (result.Succeeded) await LoadAsync();
    }
    finally
    {
        savingHeader = false;
    }
}

private async Task PublishAsync()
{
    var result = await FeeScheduleService.PublishAsync(Id);
    Show(result, "Schedule published.");
    await LoadAsync();
}

private async Task UnpublishAsync()
{
    var result = await FeeScheduleService.UnpublishAsync(Id);
    Show(result, "Schedule unpublished.");
    await LoadAsync();
}

private void OpenAddItem()
{
    itemEditingId = null;
    itemForm = new ItemFormModel
    {
        DisplayOrder = (detail?.Items.Count ?? 0) + 1,
    };
    showItemForm = true;
}

private void OpenEditItem(FeeScheduleItemDto item)
{
    itemEditingId = item.Id;
    itemForm = new ItemFormModel
    {
        FeeCategoryId = item.FeeCategoryId,
        Description = item.Description,
        Amount = item.Amount,
        IsMandatory = item.IsMandatory,
        DisplayOrder = item.DisplayOrder,
    };
    showItemForm = true;
}

private async Task SaveItemAsync()
{
    if (itemForm.FeeCategoryId is null ||
string.IsNullOrEmpty(itemForm.Description)
    || itemForm.Amount <= 0)
    {

```

```

        Notification.Notify(NotificationSeverity.Warning, "Missing fields",
            "Pick a category, write a description, and set an amount.");
        return;
    }

    itemSaving = true;
    try
    {
        var result = await FeeScheduleService.UpsertItemAsync(new
UpsertFeeScheduleItemRequest
        {
            Id = itemEditingId,
            FeeScheduleId = Id,
            FeeCategoryId = itemForm.FeeCategoryId.Value,
            Description = itemForm.Description,
            Amount = itemForm.Amount,
            IsMandatory = itemForm.IsMandatory,
            DisplayOrder = itemForm.DisplayOrder,
        });
        Show(result, "Item saved.");
        if (result.Succeeded)
        {
            showItemForm = false;
            await LoadAsync();
        }
    }
    finally
    {
        itemSaving = false;
    }
}

private async Task RemoveItemAsync(FeeScheduleItemDto item)
{
    var confirm = await Dialog.Confirm($"Remove item '{item.Description}'?",
        "Remove line item",
        new ConfirmOptions { OkButtonText = "Remove", CancelButtonText = "Cancel" });
    if (confirm != true) return;

    var result = await FeeScheduleService.RemoveItemAsync(item.Id);
    Show(result, "Item removed.");
    await LoadAsync();
}

private void Show(OperationResult result, string success)
{
    if (result.Succeeded)
        Notification.Notify(NotificationSeverity.Success, "Done", success);
    else
        Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
}
}

```

9.3 Invoices.razor — the invoice list

Listing — `src/NaijaPrimeSchool.Web/Components/Pages/Finance/Invoices.razor`

```
@page "/invoices"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},{Roles.SchoolBursar}")]
@rendermode InteractiveServer

@inject IInvoiceService InvoiceService
@inject ITermService TermService
@inject ILookupService LookupService
@inject NavigationManager Nav
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Invoices · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Invoices</h1>
        <p>Every bill issued to a pupil for a term, with status, balance, and audit trail.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="Issue invoices" Icon="receipt_long"
        ButtonStyle="ButtonStyle.Primary"
        Click="@(() => Nav.NavigateTo("/invoices/issue"))" />
    </div>
</div>

<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <div class="nps-filter-bar">
        <div class="nps-field">
            <label>Search</label>
            <RadzenTextBox @bind-Value="search" Placeholder="Invoice no, pupil name, admission no"
            Change="@(_ => LoadAsync())" Style="min-width: 280px;" />
        </div>
        <div class="nps-field">
            <label>Term</label>
            <RadzenDropDown TValue="Guid?" Data="@terms" @bind-Value="termId"
            ValueProperty="Id" TextProperty="TermTypeName"
            AllowClear="true" Placeholder="All terms"
            Change="@(_ => LoadAsync())" Style="min-width: 220px;">
                <Template Context="t">
                    @{ var dto = (TermDto)t; }
                    @dto.TermTypeName - @dto.SessionName
                </Template>
            </RadzenDropDown>
        </div>
        <div class="nps-field">
            <label>Status</label>
            <RadzenDropDown TValue="Guid?" Data="@statuses" TextProperty="Name"
            ValueProperty="Id">
```

```

        @bind-Value="statusId" AllowClear="true" Placeholder="All
statuses"
        Change="@(_ => LoadAsync())" Style="min-width: 200px;" />
    </div>
</div>
</RadzenCard>

<RadzenCard class="nps-card">
    <RadzenDataGrid TItem="InvoiceDto" Data="@invoices" IsLoading="@loading"
        AllowPaging="true" PageSize="25" EmptyText="No invoices match your
filters.">
        <Columns>
            <RadzenDataGridColumn TItem="InvoiceDto" Title="Invoice">
                <Template Context="i">
                    <strong>@i.InvoiceNumber</strong>
                    <br />
                    <small style="color: var(--nps-ink-500);">Issued
@i.IssuedOn.ToString("d MMM yyyy")</small>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="InvoiceDto" Title="Pupil">
                <Template Context="i">
                    <div class="nps-cell-with-avatar">
                        <StudentAvatar PhotoUrl="@i.StudentPhotoUrl"
                            FirstName="@i.StudentFirstName"
                            LastName="@i.StudentLastName" />
                        <div style="display:flex; flex-direction:column;">
                            <strong>@i.StudentName</strong>
                            <small style="color: var(--nps-
ink-500);">@i.StudentAdmissionNumber · @i.SchoolClassName</small>
                        </div>
                    </div>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="InvoiceDto" Title="Term" Property="TermName" />
            <RadzenDataGridColumn TItem="InvoiceDto" Title="Total" Width="120px">
                <Template Context="i">@i.AmountDue.ToString("N2")</Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="InvoiceDto" Title="Paid" Width="120px">
                <Template Context="i">@i.AmountPaid.ToString("N2")</Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="InvoiceDto" Title="Balance" Width="120px">
                <Template Context="i">
                    <strong>@i.Balance.ToString("N2")</strong>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="InvoiceDto" Title="Status" Width="140px">
                <Template Context="i">
                    <RadzenBadge Text="@i.InvoiceStatusName"
                        BadgeStyle="@StatusStyle(i.InvoiceStatusCode)"
                        Variant="Variant.Flat" />
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="InvoiceDto" Title="Actions" Width="140px"
                Sortable="false" TextAlign="TextAlign.Right">

```



```

        <Template Context="i">
            <div class="nps-inline-actions">
                <RadzenButton Icon="visibility" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                ButtonStyle="ButtonStyle.Primary" title="View"
                                Click="@(() =>
Nav.NavigateTo($"invoices/{i.Id}"))" />
                <RadzenButton Icon="payments" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                ButtonStyle="ButtonStyle.Success" title="Record
payment"
                                Click="@(() => Nav.NavigateTo($"payments/new?
studentId={i.StudentId}&invoiceId={i.Id}"))" />
            </div>
        </Template>
    </RadzenDataGridColumn>
</Columns>
</RadzenDataGrid>
</RadzenCard>

```

```

@code {
    private IReadOnlyList<InvoiceDto> invoices = [];
    private List<TermDto> terms = [];
    private IReadOnlyList<LookupDto> statuses = [];

    private bool loading;
    private string? search;
    private Guid? termId;
    private Guid? statusId;

    protected override async Task OnInitializedAsync()
    {
        terms = (await TermService.ListAsync()).ToList();
        termId = terms.FirstOrDefault(t => t.IsCurrent)?.Id;
        statuses = await LookupService.GetInvoiceStatusesAsync();
        await LoadAsync();
    }

    private async Task LoadAsync()
    {
        loading = true;
        try
        {
            invoices = await InvoiceService.ListAsync(new InvoiceFilter
            {
                TermId = termId,
                InvoiceStatusId = statusId,
                Search = search,
            });
        }
        finally
        {
            loading = false;
            StateHasChanged();
        }
    }
}

```

```

    }
}

private static BadgeStyle StatusStyle(string code) => code switch
{
    "PAID" => BadgeStyle.Success,
    "PARTIAL" => BadgeStyle.Info,
    "OVERDUE" => BadgeStyle.Danger,
    "CANCELLED" => BadgeStyle.Secondary,
    _ => BadgeStyle.Warning,
};
}

```

9.4 IssueInvoices.razor — fan-out from a schedule

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Finance/IssueInvoices.razor

```

@page "/invoices/issue"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},{Roles.SchoolBursar}")]
@rendermode InteractiveServer

@Inject IFeeScheduleService FeeScheduleService
@Inject IInvoiceService InvoiceService
@Inject ITermService TermService
@Inject ILookupService LookupService
@Inject NavigationManager Nav
@Inject NotificationService Notification

<PageTitle>Issue invoices · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Issue invoices</h1>
        <p>Pick a published fee schedule and a class. The system creates one invoice per actively-enrolled pupil with the schedule's line items.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="Back" Icon="arrow_back" Variant="Variant.Flat"
            ButtonStyle="ButtonStyle.Light" Click="@(() =>
Nav.NavigateTo("/invoices"))" />
    </div>
</div>

<RadzenCard class="nps-card">
    <div class="nps-form-grid">
        <div class="nps-field">
            <label>Term *</label>
            <RadzenDropDown TValue="Guid?" Data="@terms" @bind-Value="termId"
                ValueProperty="Id" TextProperty="TermTypeName"
                Placeholder="Pick a term"
                Change="@(_ => ReloadSchedulesAsync())">
                <Template Context="t">
                    @{ var dto = (TermDto)t; }

```

```

        @dto.TermTypeName - @dto.SessionName
    </Template>
</RadzenDropDown>
</div>
<div class="nps-field">
    <label>Fee schedule *</label>
    <RadzenDropDown TValue="Guid?" Data="@schedules" TextProperty="Title"
ValueProperty="Id"
        @bind-Value="scheduleId" Placeholder="Pick a published
schedule" />
    <small style="color: var(--nps-ink-500);">
        Only published schedules for the chosen term are listed.
    </small>
</div>
<div class="nps-field">
    <label>Class *</label>
    <RadzenDropDown TValue="Guid?" Data="@classes" TextProperty="Name"
ValueProperty="Id"
        @bind-Value="classId" Placeholder="Pick a class" />
    <small style="color: var(--nps-ink-500);">
        Must match the class level of the chosen schedule.
    </small>
</div>
<div class="nps-field">
    <label>Issued on *</label>
    <RadzenDatePicker TValue="DateTime?" @bind-Value="issuedOn"
        DateFormat="d MMM yyyy" ShowTime="false" />
</div>
<div class="nps-field">
    <label>Due date</label>
    <RadzenDatePicker TValue="DateTime?" @bind-Value="dueDate"
        DateFormat="d MMM yyyy" ShowTime="false" />
</div>
</div>
<div class="nps-form-actions">
    <RadzenButton Text="@(!issuing ? "Issuing..." : "Issue invoices")"
        Icon="receipt_long" ButtonStyle="ButtonStyle.Primary"
        Disabled="@issuing" Click="@IssueAsync" />
</div>
</RadzenCard>

@code {
    private List<TermDto> terms = [];
    private List<FeeScheduleDto> schedules = [];
    private IReadOnlyList<LookupDto> classes = [];

    private Guid? termId;
    private Guid? scheduleId;
    private Guid? classId;
    private DateTime? issuedOn = DateTime.Today;
    private DateTime? dueDate;

    private bool issuing;

```

```

protected override async Task OnInitializedAsync()
{
    terms = (await TermService.ListAsync()).ToList();
    termId = terms.FirstOrDefault(t => t.IsCurrent)?.Id;
    classes = await LookupService.GetClassesForSessionAsync();
    await ReloadSchedulesAsync();
}

private async Task ReloadSchedulesAsync()
{
    scheduleId = null;
    if (termId.HasValue)
    {
        schedules = (await FeeScheduleService.ListAsync(new FeeScheduleFilter
        {
            TermId = termId,
            IsPublished = true,
        })).ToList();
    }
    else
    {
        schedules = [];
    }
}

private async Task IssueAsync()
{
    if (scheduleId is null || classId is null || issuedOn is null)
    {
        Notification.Notify(NotificationSeverity.Warning, "Missing fields",
            "Pick a schedule, a class, and an issued-on date.");
        return;
    }

    issuing = true;
    try
    {
        var result = await InvoiceService.IssueAsync(new IssueInvoicesRequest
        {
            FeeScheduleId = scheduleId.Value,
            SchoolClassId = classId.Value,
            IssuedOn = DateOnly.FromDateTime(issuedOn.Value),
            DueDate = dueDate.HasValue ? DateOnly.FromDateTime(dueDate.Value) : null,
        });

        if (result.Succeeded && result.Data is not null)
        {
            var msg = $"Issued {result.Data.Issued} invoice(s)";
            if (result.Data.Skipped > 0) msg += $" ; skipped {result.Data.Skipped}";
            if (result.Data.Warnings.Count > 0)
                msg += $" ({result.Data.Warnings.Count} warning(s))";
            Notification.Notify(NotificationSeverity.Success, "Done", msg);
            Nav.NavigateTo("/invoices");
        }
    }
}

```

```

        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed",
                string.Join("; ", result.Errors));
        }
    }
    finally
    {
        issuing = false;
    }
}
}

```

9.5 InvoiceDetail.razor — the invoice view

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Finance/InvoiceDetail.razor

```

@page "/invoices/{Id:guid}"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},
{Roles.SchoolBursar}")]]
@rendermode InteractiveServer

@inject IInvoiceService InvoiceService
@inject NavigationManager Nav
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Invoice · Naija Prime School</PageTitle>

@if (loading)
{
    <p>Loading invoice...</p>
}
else if (detail is null)
{
    <RadzenAlert Severity="AlertSeverity.Warning">Invoice not found.</RadzenAlert>
}
else
{
    <div class="nps-page-header">
        <div class="nps-card-headline">
            <StudentAvatar PhotoUrl="@detail.Invoice.StudentPhotoUrl"
                FirstName="@detail.Invoice.StudentFirstName"
                LastName="@detail.Invoice.StudentLastName"
                Size="large" />

            <div>
                <h1>@detail.Invoice.InvoiceNumber</h1>
                <p>
                    @detail.Invoice.StudentName · @detail.Invoice.StudentAdmissionNumber ·
                    @detail.Invoice.SchoolClassName · @detail.Invoice.TermName
                </p>
            </div>
        </div>
        <div class="nps-page-actions">

```

```

        <RadzenBadge Text="@detail.Invoice.InvoiceStatusName"
            BadgeStyle="@StatusStyle(detail.Invoice.InvoiceStatusCode)"
            Variant="Variant.Flat" />
        <RadzenButton Text="Record payment" Icon="payments"
            ButtonStyle="ButtonStyle.Primary"
            Click="@GoToRecordPayment" />
        <RadzenButton Text="Back" Icon="arrow_back" Variant="Variant.Flat"
            ButtonStyle="ButtonStyle.Light" Click="@(() =>
            Nav.NavigateTo("/invoices"))" />
    </div>
</div>

<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <div class="nps-stat-grid">
        <div class="nps-stat-card">
            <div class="nps-stat-icon"><RadzenIcon Icon="request_quote" /></div>
            <div>
                <div class="nps-stat-label">Subtotal</div>
                <div
                    class="nps-stat-value">@detail.Invoice.Subtotal.ToString("N2")</div>
            </div>
        </div>
        <div class="nps-stat-card">
            <div class="nps-stat-icon"><RadzenIcon Icon="discount" /></div>
            <div>
                <div class="nps-stat-label">Discount</div>
                <div class="nps-stat-
                    value">@detail.Invoice.DiscountTotal.ToString("N2")</div>
            </div>
        </div>
        <div class="nps-stat-card">
            <div class="nps-stat-icon"><RadzenIcon Icon="paid" /></div>
            <div>
                <div class="nps-stat-label">Paid</div>
                <div class="nps-stat-
                    value">@detail.Invoice.AmountPaid.ToString("N2")</div>
            </div>
        </div>
        <div class="nps-stat-card">
            <div class="nps-stat-icon"><RadzenIcon Icon="account_balance_wallet"
                /></div>
            <div>
                <div class="nps-stat-label">Balance</div>
                <div
                    class="nps-stat-value">@detail.Invoice.Balance.ToString("N2")</div>
            </div>
        </div>
    </div>
</RadzenCard>

<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <h3 style="margin: 0 0 1rem;">Line items</h3>
    <RadzenDataGrid TItem="InvoiceLineDto" Data="@detail.Lines" AllowPaging="false">
        <Columns>
            <RadzenDataGridColumn TItem="InvoiceLineDto" Title="Category"

```

```

Property="FeeCategoryName" Width="180px" />
    <RadzenDataGridColumn TItem="InvoiceLineDto" Title="Description"
Property="Description" />
    <RadzenDataGridColumn TItem="InvoiceLineDto" Title="Amount" Width="140px">
        <Template Context="1">@1.Amount.ToString("N2")</Template>
    </RadzenDataGridColumn>
    <RadzenDataGridColumn TItem="InvoiceLineDto" Title="Discount"
Width="180px">
        <Template Context="1">
            <RadzenNumeric TValue="decimal" Value="@1.Discount" Min="0"
Max="@1.Amount" Format="N2"
            Disabled="@((detail.Invoice.InvoiceStatusCode ==
"CANCELLED"))"
            Change="@((v => UpdateDiscountAsync(1.Id, v)))" />
        </Template>
    </RadzenDataGridColumn>
    <RadzenDataGridColumn TItem="InvoiceLineDto" Title="Net" Width="140px">
        <Template
Context="1"><strong>@1.LineTotal.ToString("N2")</strong></Template>
    </RadzenDataGridColumn>
</Columns>
</RadzenDataGrid>
</RadzenCard>

<RadzenCard class="nps-card">
    <h3 style="margin: 0 0 1rem;">Payments applied (@detail.Payments.Count)</h3>
    @if (detail.Payments.Count == 0)
    {
        <p style="color: var(--nps-ink-500);">No payments yet.</p>
    }
    else
    {
        <RadzenDataGrid TItem="InvoicePaymentDto" Data="@detail.Payments"
AllowPaging="false">
            <Columns>
                <RadzenDataGridColumn TItem="InvoicePaymentDto" Title="Receipt">
                    <Template Context="p">
                        <strong>@p.ReceiptNumber</strong>
                        <br />
                        <small style="color: var(--nps-ink-500);">@p.PaidOn.ToString("d
MMM yyyy")</small>
                    </Template>
                </RadzenDataGridColumn>
                <RadzenDataGridColumn TItem="InvoicePaymentDto" Title="Method"
Property="PaymentMethodName" />
                <RadzenDataGridColumn TItem="InvoicePaymentDto" Title="Reference"
Property="Reference" />
                <RadzenDataGridColumn TItem="InvoicePaymentDto" Title="Amount applied"
Width="160px">
                    <Template
Context="p"><strong>@p.AmountApplied.ToString("N2")</strong></Template>
                </RadzenDataGridColumn>
                <RadzenDataGridColumn TItem="InvoicePaymentDto" Title="Open"
Width="100px" Sortable="false" TextAlign="TextAlign.Right">
                    <Template Context="p">

```

```

<RadzenButton Icon="open_in_new" Size="ButtonSize.Small"
Variant="Variant.Flat"
ButtonStyle="ButtonStyle.Light"
Click="@(() =>
Nav.NavigateTo($"payments/{p.PaymentId}"))" />
</Template>
</RadzenDataGridColumn>
</Columns>
</RadzenDataGrid>
}
</RadzenCard>

@if (!string.IsNullOrEmpty(detail.Invoice.Notes))
{
    <RadzenCard class="nps-card" Style="margin-top:1rem;">
        <h3 style="margin: 0 0 0.5rem;">Notes</h3>
        <p style="margin: 0;">@detail.Invoice.Notes</p>
    </RadzenCard>
}

}

@code {
    [Parameter] public Guid Id { get; set; }

    private InvoiceDetailDto? detail;
    private bool loading = true;

    protected override Task OnInitializedAsync() => LoadAsync();

    private async Task LoadAsync()
    {
        loading = true;
        try { detail = await InvoiceService.GetByIdAsync(Id); }
        finally { loading = false; }
    }

    private void GoToRecordPayment()
    {
        if (detail is null) return;
        Nav.NavigateTo($"payments/new?studentId={detail.Invoice.StudentId}
&invoiceId={Id}");
    }

    private async Task UpdateDiscountAsync(Guid lineId, decimal discount)
    {
        var result = await InvoiceService.SetLineDiscountAsync(new
UpdateInvoiceLineDiscountRequest
        {
            InvoiceLineId = lineId,
            Discount = discount,
        });
        if (result.Succeeded)
        {
            Notification.Notify(NotificationSeverity.Success, "Saved", "Discount

```



```

applied.");
        await LoadAsync();
    }
    else
    {
        Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
        await LoadAsync();
    }
}

private static BadgeStyle StatusStyle(string code) => code switch
{
    "PAID" => BadgeStyle.Success,
    "PARTIAL" => BadgeStyle.Info,
    "OVERDUE" => BadgeStyle.Danger,
    "CANCELLED" => BadgeStyle.Secondary,
    _ => BadgeStyle.Warning,
};
}

```

9.6 Payments.razor — the receipts list

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Finance/Payments.razor

```

@page "/payments"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},
{Roles.SchoolBursar}")]]
@rendermode InteractiveServer

@inject IPaymentService PaymentService
@inject ILookupService LookupService
@inject NavigationManager Nav
@inject NotificationService Notification

<PageTitle>Payments · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Payments</h1>
        <p>Every receipt issued for a pupil's payment. Filter by method, status, date
range, or pupil.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="Record payment" Icon="add" ButtonStyle="ButtonStyle.Primary"
Click="@(() => Nav.NavigateTo("/payments/new"))" />
    </div>
</div>

<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <div class="nps-filter-bar">
        <div class="nps-field">
            <label>Search</label>
            <RadzenTextBox @bind-Value="search" Placeholder="Receipt no, pupil, reference"

```

[illegible]

```

ink-500);">@p.StudentAdmissionNumber</small>
        </div>
    </div>
</Template>
</RadzenDataGridColumn>
<RadzenDataGridColumn TItem="PaymentDto" Title="Method"
Property="PaymentMethodName" Width="160px" />
<RadzenDataGridColumn TItem="PaymentDto" Title="Reference" Property="Reference"
/>

<RadzenDataGridColumn TItem="PaymentDto" Title="Amount" Width="120px">
    <Template Context="p"><strong>@p.Amount.ToString("N2")</strong></Template>
</RadzenDataGridColumn>
<RadzenDataGridColumn TItem="PaymentDto" Title="Allocated" Width="120px">
    <Template Context="p">@p.AmountAllocated.ToString("N2")</Template>
</RadzenDataGridColumn>
<RadzenDataGridColumn TItem="PaymentDto" Title="Status" Width="140px">
    <Template Context="p">
        <RadzenBadge Text="@p.PaymentStatusName"
            BadgeStyle="@StatusStyle(p.PaymentStatusCode)"
            Variant="Variant.Flat" />
    </Template>
</RadzenDataGridColumn>
<RadzenDataGridColumn TItem="PaymentDto" Title="Open" Width="100px"
Sortable="false" TextAlign="TextAlign.Right">
    <Template Context="p">
        <RadzenButton Icon="visibility" Size="ButtonSize.Small"
Variant="Variant.Flat"
            ButtonStyle="ButtonStyle.Light"
            Click="@(() => Nav.NavigateTo($"payments/{p.Id}"))" />
    </Template>
</RadzenDataGridColumn>
</Columns>
</RadzenDataGrid>
</RadzenCard>

@code {
    private IReadOnlyList<PaymentDto> payments = [];
    private IReadOnlyList<LookupDto> methods = [];
    private IReadOnlyList<LookupDto> statuses = [];
    private bool loading;

    private string? search;
    private Guid? methodId;
    private Guid? statusId;
    private DateTime? fromDate;
    private DateTime? toDate;

    protected override async Task OnInitializedAsync()
    {
        methods = await LookupService.GetPaymentMethodsAsync();
        statuses = await LookupService.GetPaymentStatusesAsync();
        await LoadAsync();
    }
}

```

```

private async Task LoadAsync()
{
    loading = true;
    try
    {
        payments = await PaymentService.ListAsync(new PaymentFilter
        {
            Search = search,
            PaymentMethodId = methodId,
            PaymentStatusId = statusId,
            FromDate = fromDate.HasValue ? DateOnly.FromDateTime(fromDate.Value) :
null,
            ToDate = toDate.HasValue ? DateOnly.FromDateTime(toDate.Value) : null,
        });
    }
    finally
    {
        loading = false;
        StateHasChanged();
    }
}

private static BadgeStyle StatusStyle(string code) => code switch
{
    "CONFIRMED" => BadgeStyle.Success,
    "PENDING" => BadgeStyle.Warning,
    "REFUNDED" => BadgeStyle.Secondary,
    "BOUNCED" => BadgeStyle.Danger,
    _ => BadgeStyle.Info,
};
}

```

9.7 RecordPayment.razor — the bursar's collection workflow

The most complex page in the sprint. Picking a pupil reloads their ledger of outstanding invoices; the bursar then either auto-allocates oldest-first or hand-types per-row amounts. The total allocated, the amount unallocated, and a live running tally are visible above the table so the bursar can sanity-check the math before saving.

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Finance/RecordPayment.razor

```

@page "/payments/new"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},{Roles.SchoolBursar}")]
@rendermode InteractiveServer

@inject IPaymentService PaymentService
@inject IInvoiceService InvoiceService
@inject ILookupService LookupService
@inject NavigationManager Nav
@inject NotificationService Notification

<PageTitle>Record payment · Naija Prime School</PageTitle>

```

```

<div class="nps-page-header">
    <div>
        <h1>Record payment</h1>
        <p>Pick a pupil, enter the payment, and allocate it across one or more outstanding
invoices.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="Back" Icon="arrow_back" Variant="Variant.Flat"
            ButtonStyle="ButtonStyle.Light" Click="@(() =>
Nav.NavigateTo("/payments"))" />
    </div>
</div>

<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <h3 style="margin: 0 0 1rem;">Payment details</h3>
    <div class="nps-form-grid">
        <div class="nps-field nps-span-2">
            <label>Pupil *</label>
            <RadzenDropDown TValue="Guid?" Data="@students" TextProperty="Name"
ValueProperty="Id"
                @bind-Value="studentId" Placeholder="Search and pick a pupil"
                AllowFiltering="true"
                FilterCaseSensitivity="FilterCaseSensitivity.CaseInsensitive"
                Change="@(_ => StudentChangedAsync())" />
        </div>
        <div class="nps-field">
            <label>Payment date *</label>
            <RadzenDatePicker TValue="DateTime?" @bind-Value="paidOn"
                DateFormat="d MMM yyyy" ShowTime="false" />
        </div>
        <div class="nps-field">
            <label>Method *</label>
            <RadzenDropDown TValue="Guid?" Data="@methods" TextProperty="Name"
ValueProperty="Id"
                @bind-Value="methodId" Placeholder="Pick a method" />
        </div>
        <div class="nps-field">
            <label>Amount *</label>
            <RadzenNumeric TValue="decimal" @bind-Value="amount" Min="0" Format="N2" />
        </div>
        <div class="nps-field">
            <label>Reference</label>
            <RadzenTextBox @bind-Value="reference" Placeholder="Cheque no, bank ref,
transaction id" />
        </div>
        <div class="nps-field nps-span-2">
            <label>Notes</label>
            <RadzenTextArea @bind-Value="notes" Rows="2" />
        </div>
    </div>
</RadzenCard>

@if (ledger is not null && ledger.Invoices.Any(i => i.Balance > 0))
{
    <RadzenCard class="nps-card">

```

```

<h3 style="margin: 0 0 1rem;">Outstanding invoices</h3>
<p style="color: var(--nps-ink-500); margin: 0 0 1rem;">
    Total entered: <strong>@amount.ToString("N2")</strong> .
    Total allocated: <strong>@TotalAllocated.ToString("N2")</strong> .
    Unallocated: <strong>@((amount - TotalAllocated).ToString("N2"))</strong>
</p>
<table class="nps-score-grid">
    <thead>
        <tr>
            <th>Invoice</th>
            <th>Term</th>
            <th style="width: 140px; text-align: right;">Balance</th>
            <th style="width: 200px;">Allocate</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var inv in ledger.Invoices.Where(i => i.Balance > 0)
                                .OrderBy(i => i.IssuedOn))
        {
            var iv = inv;
            <tr>
                <td>
                    <strong>@iv.InvoiceNumber</strong>
                    <br />
                    <small style="color: var(--nps-ink-500);">
                        @iv.SchoolClassName · issued @iv.IssuedOn.ToString("d MMM
yyyyy")
                    </small>
                </td>
                <td>@iv.TermName</td>
                <td style="text-align:
right;"><strong>@iv.Balance.ToString("N2")</strong></td>
                <td>
                    <RadzenNumeric TValue="decimal" Value="@GetAllocation(iv.Id)"
                        Min="0" Max="@iv.Balance" Format="N2"
                        Change="@((decimal v) => SetAllocation(iv.Id,
v))" />
                </td>
            </tr>
        }
    </tbody>
</table>
<div class="nps-form-actions">
    <RadzenButton Text="Auto-allocate oldest first" Icon="auto_fix_high"
Variant="Variant.Flat"
                ButtonStyle="ButtonStyle.Info" Click="@AutoAllocate" />
    <RadzenButton Text="@((saving ? "Saving..." : "Save payment"))"
                Icon="save" ButtonStyle="ButtonStyle.Primary"
                Disabled="@saving" Click="@SaveAsync" />
</div>
</RadzenCard>
}
else if (studentId.HasValue)
{
    <RadzenCard class="nps-card">

```

```

        <p style="color: var(--nps-ink-500);">
            This pupil has no outstanding invoices. You can still record an unallocated
payment
            (credit on account) by leaving the table empty.
        </p>
        <div class="nps-form-actions">
            <RadzenButton Text="@{(saving ? "Saving..." : "Save payment")}"
                Icon="save" ButtonStyle="ButtonStyle.Primary"
                Disabled="@saving" Click="@SaveAsync" />
        </div>
    </RadzenCard>
}

@code {
    [SupplyParameterFromQuery] public Guid? StudentIdParam { get; set; }
    [SupplyParameterFromQuery(Name = "studentId")] public Guid? QueryStudentId { get;
set; }
    [SupplyParameterFromQuery(Name = "invoiceId")] public Guid? QueryInvoiceId { get;
set; }

    private IReadOnlyList<LookupDto> students = [];
    private IReadOnlyList<LookupDto> methods = [];

    private Guid? studentId;
    private Guid? methodId;
    private DateTime? paidOn = DateTime.Today;
    private decimal amount;
    private string? reference;
    private string? notes;
    private bool saving;

    private StudentLedgerDto? ledger;
    private Dictionary<Guid, decimal> allocations = new();

    private decimal TotalAllocated => allocations.Values.Sum();

    protected override async Task OnInitializedAsync()
    {
        methods = await LookupService.GetPaymentMethodsAsync();
        students = await LookupService.GetStudentsAsync();
        if (QueryStudentId.HasValue)
        {
            studentId = QueryStudentId;
            await StudentChangedAsync();
            if (QueryInvoiceId.HasValue
                && ledger is not null
                && ledger.Invoices.FirstOrDefault(i => i.Id == QueryInvoiceId.Value) is { }
invoice
                && invoice.Balance > 0)
            {
                allocations[invoice.Id] = invoice.Balance;
                amount = invoice.Balance;
            }
        }
    }
}

```

```

}

private async Task StudentChangedAsync()
{
    allocations = new Dictionary<Guid, decimal>();
    if (studentId is null)
    {
        ledger = null;
        return;
    }
    ledger = await InvoiceService.GetStudentLedgerAsync(studentId.Value);
}

private decimal GetAllocation(Guid invoiceId) =>
    allocations.TryGetValue(invoiceId, out var v) ? v : 0m;

private void SetAllocation(Guid invoiceId, decimal value)
{
    if (value <= 0m) allocations.Remove(invoiceId);
    else allocations[invoiceId] = value;
}

private void AutoAllocate()
{
    if (ledger is null) return;

    allocations = new Dictionary<Guid, decimal>();
    var remaining = amount;
    foreach (var invoice in ledger.Invoices
        .Where(i => i.Balance > 0)
        .OrderBy(i => i.IssuedOn))
    {
        if (remaining <= 0) break;
        var take = Math.Min(invoice.Balance, remaining);
        allocations[invoice.Id] = take;
        remaining -= take;
    }
}

private async Task SaveAsync()
{
    if (studentId is null || methodId is null || paidOn is null || amount <= 0)
    {
        Notification.Notify(NotificationSeverity.Warning, "Missing fields",
            "Pick a pupil, method, date, and an amount greater than zero.");
        return;
    }

    if (TotalAllocated > amount)
    {
        Notification.Notify(NotificationSeverity.Warning, "Over-allocated",
            "Allocations exceed the payment amount.");
        return;
    }
}

```



```

        saving = true;
        try
        {
            var request = new RecordPaymentRequest
            {
                StudentId = studentId.Value,
                PaymentMethodId = methodId.Value,
                PaidOn = DateOnly.FromDateTime(paidOn.Value),
                Amount = amount,
                Reference = reference,
                Notes = notes,
                Allocations = allocations
                    .Where(kv => kv.Value > 0m)
                    .Select(kv => new AllocationLineRequest
                    {
                        InvoiceId = kv.Key,
                        AmountApplied = kv.Value,
                    })
                    .ToList(),
            };

            var result = await PaymentService.RecordAsync(request);
            if (result.Succeeded)
            {
                Notification.Notify(NotificationSeverity.Success, "Saved",
                    "Payment recorded.");
                Nav.NavigateTo($"/payments/{result.Data}");
            }
            else
            {
                Notification.Notify(NotificationSeverity.Error, "Failed",
                    string.Join("; ", result.Errors));
            }
        }
        finally
        {
            saving = false;
        }
    }
}

```

9.8 PaymentDetail.razor — the receipt view

Listing — *src/NaijaPrimeSchool.Web/Components/Pages/Finance/PaymentDetail.razor*

```

@page "/payments/{Id:guid}"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},
{Roles.SchoolBursar}")]]
@rendermode InteractiveServer

@inject IPaymentService PaymentService
@inject NavigationManager Nav
@inject DialogService Dialog

```

```

@Inject NotificationService Notification

<PageTitle>Receipt · Naija Prime School</PageTitle>

@if (loading)
{
    <p>Loading receipt...</p>
}
else if (detail is null)
{
    <RadzenAlert Severity="AlertSeverity.Warning">Payment not found.</RadzenAlert>
}
else
{
    <div class="nps-page-header">
        <div class="nps-card-headline">
            <StudentAvatar PhotoUrl="@detail.Payment.StudentPhotoUrl"
                FirstName="@detail.Payment.StudentFirstName"
                LastName="@detail.Payment.StudentLastName"
                Size="large" />

            <div>
                <h1>@detail.Payment.ReceiptNumber</h1>
                <p>
                    @detail.Payment.StudentName · @detail.Payment.StudentAdmissionNumber ·
                    paid @detail.Payment.PaidOn.ToString("d MMM yyyy")
                </p>
            </div>
        </div>
        <div class="nps-page-actions">
            <RadzenBadge Text="@detail.Payment.PaymentStatusName"
                BadgeStyle="@StatusStyle(detail.Payment.PaymentStatusCode)"
                Variant="Variant.Flat" />
            @if (detail.Payment.PaymentStatusCode != "REFUNDED")
            {
                <RadzenButton Text="Refund" Icon="undo" Variant="Variant.Flat"
                    ButtonStyle="ButtonStyle.Warning" Click="@RefundAsync" />
            }
            <RadzenButton Text="Back" Icon="arrow_back" Variant="Variant.Flat"
                ButtonStyle="ButtonStyle.Light" Click="@(() =>
Nav.NavigateTo("/payments"))" />
        </div>
    </div>

    <RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
        <div class="nps-stat-grid">
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="paid" /></div>
                <div>
                    <div class="nps-stat-label">Amount</div>
                    <div class="nps-stat-value">@detail.Payment.Amount.ToString("N2")</div>
                </div>
            </div>
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="receipt" /></div>
                <div>

```

```

        <div class="nps-stat-label">Method</div>
        <div class="nps-stat-value" style="font-size:
1rem;">@detail.Payment.PaymentMethodName</div>
    </div>
</div>
<div class="nps-stat-card">
    <div class="nps-stat-icon"><RadzenIcon Icon="qr_code_2" /></div>
    <div>
        <div class="nps-stat-label">Reference</div>
        <div class="nps-stat-value" style="font-size: 1rem;">
            @(string.IsNullOrEmpty(detail.Payment.Reference) ? "-" :
detail.Payment.Reference)
        </div>
    </div>
</div>
<div class="nps-stat-card">
    <div class="nps-stat-icon"><RadzenIcon Icon="account_balance_wallet"
/></div>
    <div>
        <div class="nps-stat-label">Unallocated</div>
        <div class="nps-stat-
value">@detail.Payment.AmountUnallocated.ToString("N2")</div>
    </div>
</div>
</div>
</RadzenCard>

<RadzenCard class="nps-card">
    <h3 style="margin: 0 0 1rem;">Allocations (@detail.Allocations.Count)</h3>
    @if (detail.Allocations.Count == 0)
    {
        <p style="color: var(--nps-ink-500);">
            No invoices were paired with this payment – held as credit on the pupil's
ledger.
        </p>
    }
    else
    {
        <RadzenDataGrid TItem="PaymentAllocationDto" Data="@detail.Allocations"
AllowPaging="false">
            <Columns>
                <RadzenDataGridColumn TItem="PaymentAllocationDto" Title="Invoice">
                    <Template Context="a"><strong>@a.InvoiceNumber</strong></Template>
                </RadzenDataGridColumn>
                <RadzenDataGridColumn TItem="PaymentAllocationDto" Title="Applied"
Width="180px">
                    <Template
Context="a"><strong>@a.AmountApplied.ToString("N2")</strong></Template>
                </RadzenDataGridColumn>
                <RadzenDataGridColumn TItem="PaymentAllocationDto" Title="Invoice
balance after" Width="200px">
                    <Template
Context="a">@a.InvoiceBalanceAfter.ToString("N2")</Template>
                </RadzenDataGridColumn>
                <RadzenDataGridColumn TItem="PaymentAllocationDto" Title="Open invoice"

```

```

Width="120px"
                Sortable="false" TextAlign="TextAlign.Right">
                <Template Context="a">
                    <RadzenButton Icon="open_in_new" Size="ButtonSize.Small"
Variant="Variant.Flat"
                    ButtonStyle="ButtonStyle.Light"
                    Click="@(() =>
Nav.NavigateTo($"invoices/{a.InvoiceId}"))" />
                </Template>
            </RadzenDataGridColumn>
        </Columns>
    </RadzenDataGrid>
}
</RadzenCard>

@if (!string.IsNullOrEmpty(detail.Payment.Notes))
{
    <RadzenCard class="nps-card" Style="margin-top:1rem;">
        <h3 style="margin: 0 0 0.5rem;">Notes</h3>
        <p style="margin: 0;">@detail.Payment.Notes</p>
    </RadzenCard>
}
}

@code {
    [Parameter] public Guid Id { get; set; }

    private PaymentDetailDto? detail;
    private bool loading = true;

    protected override Task OnInitializedAsync() => LoadAsync();

    private async Task LoadAsync()
    {
        loading = true;
        try { detail = await PaymentService.GetByIdAsync(Id); }
        finally { loading = false; }
    }

    private async Task RefundAsync()
    {
        var confirm = await Dialog.Confirm(
            "Refund this payment? The allocations will be released and the affected invoice
balances will reopen.",
            "Refund payment",
            new ConfirmOptions { OkButtonText = "Refund", CancelButtonText = "Cancel" });
        if (confirm != true) return;

        var result = await PaymentService.RefundAsync(Id);
        if (result.Succeeded)
        {
            Notification.Notify(NotificationSeverity.Success, "Refunded", "Payment
refunded.");
            await LoadAsync();
        }
    }
}

```

```

    }
    else
    {
        Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
    }
}

private static BadgeStyle StatusStyle(string code) => code switch
{
    "CONFIRMED" => BadgeStyle.Success,
    "PENDING" => BadgeStyle.Warning,
    "REFUNDED" => BadgeStyle.Secondary,
    "BOUNCED" => BadgeStyle.Danger,
    _ => BadgeStyle.Info,
};
}

```

9.9 FinanceDashboard.razor — bursar at a glance

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Finance/FinanceDashboard.razor

```

@page "/finance"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher},
{Roles.SchoolBursar}")]
@rendermode InteractiveServer

@Inject IPaymentService PaymentService
@Inject ITermService TermService
@Inject NavigationManager Nav

<PageTitle>Bursar dashboard · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Bursar dashboard</h1>
        <p>Money in, money owed, and the breakdown across methods and fee categories – at a
glance.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenDropDown TValue="Guid?" Data="@terms" @bind-Value="termId"
            ValueProperty="Id" TextProperty="TermTypeName"
            AllowClear="true" Placeholder="All terms"
            Change="@(_ => LoadAsync())" Style="min-width: 240px;">
            <Template Context="t">
                @{ var dto = (TermDto)t; }
                @dto.TermTypeName – @dto.SessionName
            </Template>
        </RadzenDropDown>
    </div>
</div>

@if (loading || summary is null)
{

```

```

    <p>Loading dashboard...</p>
}
else
{
    <RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
        <div class="nps-stat-grid">
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="request_quote" /></div>
                <div>
                    <div class="nps-stat-label">Invoiced</div>
                    <div class="nps-stat-value">@summary.TotalInvoiced.ToString("N2")</div>
                </div>
            </div>
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="paid" /></div>
                <div>
                    <div class="nps-stat-label">Collected</div>
                    <div
class="nps-stat-value">@summary.TotalCollected.ToString("N2")</div>
                    </div>
                </div>
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="account_balance_wallet"
/></div>
                <div>
                    <div class="nps-stat-label">Outstanding</div>
                    <div
class="nps-stat-value">@summary.TotalOutstanding.ToString("N2")</div>
                    </div>
                </div>
            <div class="nps-stat-card">
                <div class="nps-stat-icon"><RadzenIcon Icon="receipt_long" /></div>
                <div>
                    <div class="nps-stat-label">Invoices</div>
                    <div class="nps-stat-value">@summary.InvoiceCount</div>
                </div>
            </div>
        </div>
    </RadzenCard>

    <RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
        <h3 style="margin: 0 0 1rem;">Invoice status breakdown</h3>
        <div class="nps-stat-grid">
            <div class="nps-stat-card">
                <div class="nps-stat-icon" style="background: #d9f1e3;"><RadzenIcon
Icon="check_circle" /></div>
                <div>
                    <div class="nps-stat-label">Paid in full</div>
                    <div class="nps-stat-value">@summary.PaidInvoiceCount</div>
                </div>
            </div>
            <div class="nps-stat-card">
                <div class="nps-stat-icon" style="background: #fff3cd; color:
#b8860b;"><RadzenIcon Icon="trending_up" /></div>
                <div>

```

```

        <div class="nps-stat-label">Partially paid</div>
        <div class="nps-stat-value">@summary.PartiallyPaidInvoiceCount</div>
    </div>
</div>
<div class="nps-stat-card">
    <div class="nps-stat-icon" style="background: #fde2e2; color:
#d64545;"><RadzenIcon Icon="hourglass_empty" /></div>
    <div>
        <div class="nps-stat-label">Unpaid</div>
        <div class="nps-stat-value">@summary.UnpaidInvoiceCount</div>
    </div>
</div>
<div class="nps-stat-card">
    <div class="nps-stat-icon"><RadzenIcon Icon="payments" /></div>
    <div>
        <div class="nps-stat-label">Payments recorded</div>
        <div class="nps-stat-value">@summary.PaymentCount</div>
    </div>
</div>
</div>
</RadzenCard>

<div class="nps-form-grid">
    <RadzenCard class="nps-card">
        <h3 style="margin: 0 0 1rem;">Collected by method</h3>
        @if (summary.ByMethod.Count == 0)
        {
            <p style="color: var(--nps-ink-500);">No payments yet.</p>
        }
        else
        {
            <RadzenDataGrid TItem="MethodBreakdownDto" Data="@summary.ByMethod"
AllowPaging="false">
                <Columns>
                    <RadzenDataGridColumn TItem="MethodBreakdownDto" Title="Method"
Property="PaymentMethodName" />
                    <RadzenDataGridColumn TItem="MethodBreakdownDto" Title="Count"
Width="100px" Property="PaymentCount" />
                    <RadzenDataGridColumn TItem="MethodBreakdownDto" Title="Total"
Width="160px">
                        <Template
Context="m"><strong>@m.TotalAmount.ToString("N2")</strong></Template>
                    </RadzenDataGridColumn>
                </Columns>
            </RadzenDataGrid>
        }
    </RadzenCard>
    <RadzenCard class="nps-card">
        <h3 style="margin: 0 0 1rem;">Invoiced by category</h3>
        @if (summary.ByCategory.Count == 0)
        {
            <p style="color: var(--nps-ink-500);">No invoices yet.</p>
        }
        else
        {

```

```

        <RadzenDataGrid TItem="CategoryBreakdownDto" Data="@summary.ByCategory"
AllowPaging="false">
            <Columns>
                <RadzenDataGridColumn TItem="CategoryBreakdownDto" Title="Category"
Property="FeeCategoryName" />
                <RadzenDataGridColumn TItem="CategoryBreakdownDto" Title="Invoiced"
Width="180px">
                    <Template
Context="c"><strong>@c.TotalInvoiced.ToString("N2")</strong></Template>
                </RadzenDataGridColumn>
            </Columns>
        </RadzenDataGrid>
    }
</RadzenCard>
</div>
}

@code {
    private FinanceSummaryDto? summary;
    private List<TermDto> terms = [];
    private Guid? termId;
    private bool loading;

    protected override async Task OnInitializedAsync()
    {
        terms = (await TermService.ListAsync()).ToList();
        termId = terms.FirstOrDefault(t => t.IsCurrent)?.Id;
        await LoadAsync();
    }

    private async Task LoadAsync()
    {
        loading = true;
        try { summary = await PaymentService.GetSummaryAsync(termId); }
        finally { loading = false; StateHasChanged(); }
    }
}

```


10. Navigation, imports, and authorization

The previously-disabled 'Finance' nav placeholder is replaced with a six-item panel: Bursar dashboard, Fee schedules, Invoices, Issue invoices, Payments, Record payment. The panel is wrapped in an `AuthorizeView` that grants access to `SuperAdmin`, `HeadTeacher`, and `SchoolBursar`.

Excerpt — `NavMenu.razor`

```
        <RadzenPanelMenuItem Text="Finance" Icon="payments" Expanded="true">
            <RadzenPanelMenuItem Text="Bursar dashboard" Icon="space_dashboard"
Path="/finance" />
            <RadzenPanelMenuItem Text="Fee schedules" Icon="receipt_long"
Path="/fees" />
            <RadzenPanelMenuItem Text="Invoices" Icon="request_quote"
Path="/invoices" />
            <RadzenPanelMenuItem Text="Issue invoices" Icon="post_add"
Path="/invoices/issue" />
            <RadzenPanelMenuItem Text="Payments" Icon="payments" Path="/payments" />
            <RadzenPanelMenuItem Text="Record payment" Icon="add_card"
Path="/payments/new" />
        </RadzenPanelMenuItem>
    </Authorized>
</AuthorizeView>
```

Excerpt — `_Imports.razor`

```
@using NaijaPrimeSchool.Application.Finance
@using NaijaPrimeSchool.Application.Finance.Dtos
```

11. Lifecycle of a finance flow

11.1 Publishing a fee schedule

- HeadTeacher opens /fees, clicks New schedule.
- Picks the current Term and the target ClassLevel.
- Adds line items (Tuition ₦80,000; Development Levy ₦15,000; Examination ₦5,000; Books ₦12,000).
- Clicks Publish. The service refuses if no items exist; otherwise IsPublished flips to true and the schedule is ready to be used.

11.2 Issuing invoices

- Open /invoices/issue.
- Pick the term, the published schedule, the matching class, an issued-on date, and (optionally) a due date.
- Click Issue invoices. InvoiceService loads every actively-enrolled pupil in the class, skips pupils that already have an invoice in that (term, class), and creates a fresh Invoice + InvoiceLine[] for everyone else.
- Invoice numbers are NPS/INV/<year>/<seq>; sequence starts from the highest existing number for the year + 1.
- The Issued status is stamped from the seeded InvoiceStatus row with Code = 'ISSUED'.

11.3 Recording a payment

- A parent comes to the bursar's window with ₦60,000 cash.
- Bursar opens /payments/new, picks the pupil from the dropdown.
- The ledger of outstanding invoices loads in the table.
- Bursar enters method Cash, amount 60,000.
- Clicks Auto-allocate oldest first — the service walks the outstanding invoices oldest-first and allocates the cash row by row.
- Clicks Save payment. PaymentService validates allocations against each invoice's balance, generates the receipt number (NPS/RCP/<year>/<seq>), and calls InvoiceService.RecomputeInvoiceTotalsAsync for every touched invoice — flipping ISSUED to PARTIAL or PAID as appropriate.
- User lands on /payments/{id} — the receipt view.

11.4 Refunding a bounced cheque

- Bursar opens the original receipt, clicks Refund.
- PaymentService deletes the allocations, flips the payment status to REFUNDED, and recomputes the touched invoices.
- Invoices return to PARTIAL or ISSUED status depending on what other payments may still be applied.
- The receipt row stays on file with the Refunded badge for the audit trail.

12. Smoke-test walkthrough

12.1 Build, migrate, run

```
dotnet restore
dotnet build NaijaPrimeSchool.slnx
dotnet run --project src/NaijaPrimeSchool.Web
```

First run applies the FeesInvoicesPayments migration and seeds the four new lookup tables. Sign in as the SuperAdmin.

12.2 Verify navigation and lookups

- The Finance panel is now an active six-item dropdown rather than a disabled placeholder.
- `SELECT Name, Code FROM FeeCategories ORDER BY DisplayOrder;` shows ten rows.
- `SELECT Name, Code FROM PaymentMethods ORDER BY DisplayOrder;` shows six rows.
- `SELECT Name, Code FROM InvoiceStatuses ORDER BY DisplayOrder;` shows six rows.
- `SELECT Name, Code FROM PaymentStatuses ORDER BY DisplayOrder;` shows four rows.

12.3 End-to-end happy path

10. Finance → Fee schedules → New. Term: current. Class level: Primary 1. Title: 'Primary 1 — First Term fees'. Save.
11. Add three items (Tuition 80,000; Development Levy 15,000; Examination 5,000).
12. Click Publish.
13. Finance → Issue invoices. Pick the term, the schedule, a Primary 1 class with enrolled pupils. Issue.
14. Finance → Invoices. Confirm one invoice per actively-enrolled pupil in the chosen class. Each should show AmountDue = 100,000 and balance = 100,000.
15. Click Record payment on any invoice row.
16. Method: Cash. Amount: 50,000. Auto-allocate. Save.
17. Open the resulting receipt. Status should be Confirmed; the invoice it touched should now show 50,000 / 100,000 with a Partial badge.
18. Open /finance. Totals should reflect the issuance and the payment.

12.4 Error paths

19. Try to publish a schedule with no items. Refused.
20. Try to unpublish a schedule after invoices have been issued. Refused.
21. Try to record a payment whose allocations exceed the amount. Refused.
22. Try to record an allocation that exceeds the invoice's outstanding balance. Refused.
23. Refund a payment, then try to record allocations against the now-released invoice — the balance should reflect the released amount.

13. Troubleshooting and gotchas

13.1 'A fee schedule already exists for this (term, class level)'

The composite unique index on FeeSchedule.(TermId, ClassLevelId) is a deliberate guard: a school should not accidentally publish two competing schedules for the same audience. If you need to change a schedule, open the existing one and edit it.

13.2 'Publish the fee schedule before issuing invoices from it'

Draft schedules are deliberately not allowed to fan out into invoices — the audit trail would otherwise be confusing. Click Publish on the schedule first.

13.3 'No actively-enrolled pupils in the selected class'

An enrolment counts as active when WithdrawnOn is null. If every pupil's enrolment has been withdrawn, the issuance produces no invoices. Re-enrol pupils, or use the family pages to confirm the class's roster.

13.4 Allocations exceed the payment amount

Both the UI and the service layer guard against this. The live running total on RecordPayment shows the math; if TotalAllocated > Amount, the save button is still clickable but the service rejects the request with a friendly message.

13.5 Cannot delete a payment

If the payment still has allocations, soft-delete is refused. Refund it first — that releases the allocations and flips the status to REFUNDED — then delete. The receipt history is preserved either way.

13.6 'Invoice statuses are not seeded' / 'Cancelled status is not seeded'

The service keys off the seeded Code values. If you cleared the InvoiceStatuses table by hand, restart the app so DatabaseInitializer re-seeds it.

13.7 The IdentityRole migration warning

Same pre-existing warning that has accompanied every sprint since sprint 1. Harmless at runtime; will be addressed in a future role-administration sprint.

14. Forward-compatibility, today

- Parent and student portals (later sprint) will read Invoices and Payments via the existing `IIInvoiceService` and `IPaymentService` contracts. The `IsPublished` flag does not exist on invoices — schools issue immediately — but the `InvoiceStatus.Code` is the gate the portal will key off.
- Aged debt reports can be added without schema changes: a new DTO + a service method that groups Invoices by (`PupilId`, `OutstandingBalance`, `DaysOverdue`).
- Online payments gateway: add a new `PaymentMethod` row, plus a minimal-API endpoint that POSTs back from the gateway, calls `PaymentService.RecordAsync` with the same shape any in-person payment uses, and emails the receipt URL to the parent.
- Discounts beyond the per-line discount column can grow into a dedicated `Scholarship` table without breaking existing invoices.
- Multi-currency: a `Currency` table plus a `CurrencyId` on `Invoice` and `Payment` is the cleanest path; the decimal precision is ready.

15. Appendix — files added or changed in sprint 6

Domain layer (new)	—
src/NaijaPrimeSchool.Domain/Finance/ FeeCategory.cs	Lookup.
src/NaijaPrimeSchool.Domain/Finance/ PaymentMethod.cs	Lookup.
src/NaijaPrimeSchool.Domain/Finance/ InvoiceStatus.cs	Lookup.
src/NaijaPrimeSchool.Domain/Finance/ PaymentStatus.cs	Lookup.
src/NaijaPrimeSchool.Domain/Finance/ FeeSchedule.cs	Per-(term, level) schedule.
src/NaijaPrimeSchool.Domain/Finance/ FeeScheduleItem.cs	Line item on a schedule.
src/NaijaPrimeSchool.Domain/Finance/Invoice.cs	Pupil bill.
src/NaijaPrimeSchool.Domain/Finance/ InvoiceLine.cs	Line item on an invoice.
src/NaijaPrimeSchool.Domain/Finance/ Payment.cs	Receipt header.
src/NaijaPrimeSchool.Domain/Finance/ PaymentAllocation.cs	Payment x invoice split.
Domain layer (modified)	—
src/NaijaPrimeSchool.Domain/Family/Student.cs	Added Invoices + Payments.
src/NaijaPrimeSchool.Domain/Academics/ Term.cs	Added FeeSchedules + Invoices.
src/NaijaPrimeSchool.Domain/Academics/ SchoolClass.cs	Added Invoices.
src/NaijaPrimeSchool.Domain/Academics/ ClassLevel.cs	Added FeeSchedules.
Application layer (new)	—
src/NaijaPrimeSchool.Application/Finance/ Dtos/FeeScheduleDtos.cs	Schedule DTOs.
src/NaijaPrimeSchool.Application/Finance/Dtos/ InvoiceDtos.cs	Invoice DTOs.
src/NaijaPrimeSchool.Application/Finance/ Dtos/PaymentDtos.cs	Payment / allocation DTOs.
src/NaijaPrimeSchool.Application/Finance/ Dtos/StudentLedgerDtos.cs	Student ledger + dashboard DTOs.
src/NaijaPrimeSchool.Application/Finance/ IFeeScheduleService.cs	Schedule service contract.
src/NaijaPrimeSchool.Application/Finance/ IInvoiceService.cs	Invoice service contract.
src/NaijaPrimeSchool.Application/Finance/ IPaymentService.cs	Payment service contract.
Application layer (modified)	—
src/NaijaPrimeSchool.Application/Users/ ILookupService.cs	Added 4 new lookup methods.

Infrastructure layer (new)	—
src/NaijaPrimeSchool.Infrastructure/Services/FeeScheduleService.cs	Schedule + items CRUD.
src/NaijaPrimeSchool.Infrastructure/Services/InvoiceService.cs	Issue, discount, cancel, ledger, status recompute.
src/NaijaPrimeSchool.Infrastructure/Services/PaymentService.cs	Record, refund, dashboard summary.
src/NaijaPrimeSchool.Infrastructure/Persistence/Migrations/20260512224258_FeesInvoicesPayments.cs	EF migration adding 10 tables.
Infrastructure layer (modified)	—
src/NaijaPrimeSchool.Infrastructure/DependencyInjection.cs	Registered 3 new services (InvoiceService twice).
src/NaijaPrimeSchool.Infrastructure/Persistence/ApplicationDbContext.cs	Added 10 DbSet, ConfigureFinance.
src/NaijaPrimeSchool.Infrastructure/Persistence/DatabaseInitializer.cs	Seeded the 4 new lookup tables.
src/NaijaPrimeSchool.Infrastructure/Services/LookupService.cs	Added 4 new lookup methods.
Web layer (new)	—
src/NaijaPrimeSchool.Web/Components/Pages/Finance/FeeSchedules.razor	Schedule list + new form.
src/NaijaPrimeSchool.Web/Components/Pages/Finance/FeeScheduleDetail.razor	Edit schedule + items.
src/NaijaPrimeSchool.Web/Components/Pages/Finance/Invoices.razor	Invoice list.
src/NaijaPrimeSchool.Web/Components/Pages/Finance/IssueInvoices.razor	Fan out from a schedule.
src/NaijaPrimeSchool.Web/Components/Pages/Finance/InvoiceDetail.razor	Invoice view with discount edit.
src/NaijaPrimeSchool.Web/Components/Pages/Finance/Payments.razor	Payments list.
src/NaijaPrimeSchool.Web/Components/Pages/Finance/RecordPayment.razor	Pupil ledger + record + allocate.
src/NaijaPrimeSchool.Web/Components/Pages/Finance/PaymentDetail.razor	Receipt view + refund.
src/NaijaPrimeSchool.Web/Components/Pages/Finance/FinanceDashboard.razor	Bursar summary.
Web layer (modified)	—
src/NaijaPrimeSchool.Web/Components/_Imports.razor	Added Finance + Finance.Dtos usings.
src/NaijaPrimeSchool.Web/Components/Layout/NavMenu.razor	Replaced disabled Finance placeholder with full panel.
Tooling (new)	—
tools/generate_sprint6_guide.py	This document's generator.

— *End of the Sprint 6 implementation guide. With the money loop closed, the next sprint can turn to inventory and the storekeeper's workflows, confident that the financial side of the school is in good order.*